Almacenamiento de la información.

Caso práctico

Ada sabe que las aplicaciones de **BK Programación** gestionarán cada vez más información. Los datos deberán almacenarse de forma **segura, organizada y accesible**, evitando duplicidades y facilitando el trabajo de distintos usuarios y aplicaciones.

Juan y **María** deberán conocer la evolución del **almacenamiento de la información**, desde los ficheros tradicionales hasta las bases de datos actuales, y las herramientas para administrarlas eficientemente.

Los primeros sistemas utilizaban **ficheros**, pero al crecer los datos surgieron problemas de **redundancia**, inconsistencia y gestión.

Las **bases de datos** permiten organizar y gestionar información mediante un **Sistema Gestor de Bases de Datos (SGBD)**, que controla los accesos y mantiene la integridad y concurrencia de los datos.

Los datos pueden organizarse mediante distintos **modelos**, especialmente el **relacional**, y almacenarse en sistemas **centralizados o distribuidos**.

Magnific. *Magnific. Almacenamiento de la información. (CC BY-NC-SA)* (CC BY-NC-SA)

Además, debe garantizarse la **protección de datos personales** y, con grandes volúmenes de información, **Big Data** y **Business Intelligence** facilitan el análisis y la **toma de decisiones**.

Datos → almacenamiento → organización → gestión → seguridad → análisis

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Materiales formativos de FP Online propiedad del MEFPD. Aviso Legal

1.- Introducción.

Las **bases de datos** son sistemas que permiten **almacenar, organizar y gestionar información** de forma estructurada, facilitando su **consulta**, **actualización** y **uso** por distintas aplicaciones y usuarios. Su objetivo es que los datos puedan mantenerse **accesibles, coherentes y seguros** durante todo su ciclo de vida.

Los **datos** intervienen en prácticamente cualquier servicio digital. Cuando compramos por Internet, usamos una aplicación bancaria, accedemos a una plataforma educativa o hacemos una reserva, se están consultando y modificando datos almacenados previamente.

*Algunos **ejemplos** habituales son:*

- **Banco:** clientes, cuentas, movimientos y transferencias.
- **Comercio electrónico:** productos, pedidos, pagos y stock.
- **Sanidad:** pacientes, citas e historiales.
- **Educación:** alumnado, matrículas, cursos y calificaciones.
- **Aplicaciones web:** usuarios, permisos, contenidos y configuraciones.
- **Reservas:** disponibilidad, clientes y operaciones realizadas.

rawpixel.com . *Bases de datos.* (CC BY-NC-SA)

El crecimiento de los servicios digitales ha incrementado tanto el **volumen de datos** como el número de usuarios que acceden a ellos. Por eso, no basta con almacenarlos: es necesario garantizar su **integridad**, **seguridad**, **disponibilidad** y un acceso eficiente.

En el desarrollo web, los datos deben mantenerse de forma **persistente** y permitir operaciones de **consulta, inserción, modificación y eliminación**. Una gestión incorrecta puede provocar **duplicidades**, inconsistencias, pérdidas de información o accesos no autorizados.

Autoevaluación

Una plataforma de comercio electrónico gestiona clientes, productos, pedidos y pagos.

¿Qué necesidades justifican especialmente el uso de una base de

datos?

- [ ] a) Mantener relaciones entre distintos tipos de información.
- [ ] b) Conservar los datos solo mientras el usuario mantenga abierta la página.
- [ ] c) Permitir accesos simultáneos de distintos usuarios de forma controlada.
- [ ] d) Mantener la información persistente y coherente aunque la aplicación se cierre.
- [ ] e) Evitar cualquier modificación de los datos almacenados.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto
5. Incorrecto

2.- Los ficheros de información.

Caso práctico

Stockbyte (CC BY-NC-SA)

Juan y **María** revisan cómo se almacenaba la información antes de que las **bases de datos** se extendieran.

—Muchas aplicaciones trabajaban directamente con **ficheros** —comenta **Juan** —. Cada programa tenía que conocer cómo estaban organizados para poder leer o modificar sus datos.

—Y todavía se utilizan —responde **María**—, por ejemplo para **configuraciones, registros, documentos o intercambio de datos**.

Ada añade que entender su estructura ayuda a comprender cómo ha evolucionado el almacenamiento de información y por qué surgieron soluciones más avanzadas.

Un **fichero o archivo** es una unidad de almacenamiento que reúne **datos relacionados** y los conserva de forma persistente. Su información puede organizarse en **registros** y **campos**, que permiten estructurar los datos para que una aplicación pueda interpretarlos y procesarlos.

Los **ficheros** siguen utilizándose actualmente para almacenar configuraciones, documentos, registros de actividad o datos que no necesitan ser gestionados mediante un SGBD.

Organización de la información

La estructura puede representarse de forma sencilla:

Fichero → Registros → Campos

Elaboración propia. *Fichero de información.* (CC BY-NC-SA)

Elemento	Función
Fichero	Contiene el conjunto de **datos relacionados**.

Elemento	Función
Registro lógico	Agrupa los datos de un mismo elemento.
Campo	Contiene un **dato concreto** dentro del registro.

Ejemplo:

En un fichero de usuarios, cada usuario puede constituir un **registro**, mientras que su **ID, nombre y correo** serían campos.

Los datos pueden ser **consultados, añadidos, modificados o eliminados** según el formato y el sistema que los gestione.

Registros físicos y bloques

El sistema de almacenamiento no siempre lee un único registro cada vez. Para mejorar la eficiencia puede transferir varios registros agrupados en un **bloque**.

- **Registro lógico:** unidad de información utilizada por la aplicación.
- **Bloque o registro físico:** conjunto de datos transferidos entre el **almacenamiento** y la **memoria** en una operación.
- **Factor de blocaje:** número de **registros lógicos** incluidos en un bloque.

Si un bloque contiene **8 registros**, su **factor de bloqueo es 8**.

Reflexiona

Una aplicación almacena la información de sus clientes en un fichero. Cada cliente ocupa un **registro lógico**, pero el sistema agrupa varios registros dentro de un mismo **bloque físico** antes de transferirlos entre el almacenamiento y la memoria.

¿Qué ventaja puede tener agrupar varios registros en un bloque y qué ocurriría con el factor de blocaje si cada bloque pasara de contener 5 registros a contener 10?

Mostrar retroalimentación

Agrupar varios **registros lógicos** en un mismo **bloque físico** permite reducir el número de operaciones de lectura y escritura necesarias para procesar la información. Si un bloque pasa de contener 5 registros a contener 10, el **factor de blocaje** aumenta de **5 a 10**, ya que este valor indica cuántos registros lógicos se agrupan en cada bloque.

2.1.- Tipos de ficheros.

Los **ficheros** pueden clasificarse según la **función que desempeñan** dentro de una aplicación. De forma general, se distinguen los **ficheros permanentes**, que conservan información durante periodos prolongados, y los **ficheros temporales**, creados para tareas concretas y de corta duración.

Según la función que vaya a desempeñar los ficheros, éstos pueden ser clasificados de varias maneras. En la siguiente imagen puedes observar una posible clasificación.

Elaboración propia. *Tipos de ficheros según su organización.* (CC BY-NC-SA)

Ficheros permanentes

Mantienen información durante un periodo prolongado y forman parte de los **datos habituales de la aplicación**.

Tipo	Función	Ejemplo
Maestros	Contienen los **datos principales y actuales**, que pueden modificarse.	Catálogo de productos de una tienda.
Constantes	Guardan datos relativamente **estables**, utilizados principalmente para consulta.	Tabla de países o provincias.
Históricos	Conservan información que dejó de ser actual, pero debe mantenerse como **historial**.	Pedidos cerrados de años anteriores.

Ficheros temporales

Se crean para una **tarea concreta** y normalmente dejan de ser necesarios cuando termina el proceso.

Tipo	Función	Ejemplo
Intermedios	Guardan resultados que serán utilizados posteriormente por **otro proceso**.	Fichero generado por una aplicación para que otra lo importe.

Tipo	Función	Ejemplo
De maniobras	Almacenan temporalmente datos auxiliares necesarios durante un **procesamiento**.	Fichero temporal creado al ordenar una gran cantidad de registros.
De resultados	Guardan los datos obtenidos por un proceso antes de su salida o utilización.	Fichero generado con el resultado de un informe.

Esta clasificación tiene principalmente un valor **funcional**: un fichero no se clasifica por su formato o extensión, sino por la función que vayan a desempeñar, **los ficheros pueden clasificarse de varias maneras.**

Actividad desplegable

Una aplicación de gestión comercial utiliza los siguientes ficheros. Relaciona cada situación con el tipo correspondiente.

Situación	Tipo de fichero
1. Contiene el catálogo actual de productos y sus precios.	
2. Guarda pedidos finalizados que deben conservarse durante varios años.	
3. Almacena temporalmente los datos obtenidos por un proceso que utilizará otro proceso posterior.	
4. Contiene una relación de provincias utilizada habitualmente para realizar consultas.	

Comprobar | Mostrar retroalimentación

1. **Maestro:** contiene los **datos actuales principales** de la aplicación y puede sufrir modificaciones.
2. **Histórico:** conserva información que ya no pertenece a la situación actual, pero debe mantenerse para su **consulta posterior**.
3. **Intermedio:** almacena temporalmente resultados que serán utilizados como **entrada de otro proceso**.

4. **Constante:** contiene datos relativamente **estables** que se utilizan principalmente para consulta.

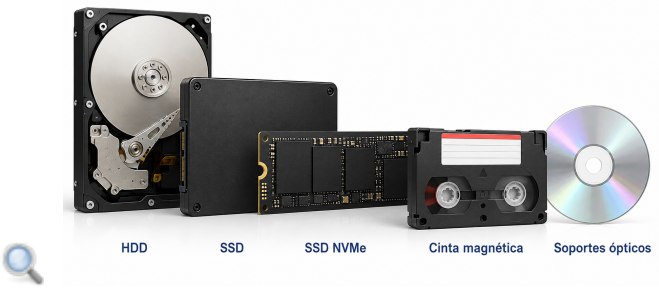
2.2.- Los soportes de información.

Los **soportes de información** son los medios físicos o tecnológicos utilizados para **almacenar datos** de forma persistente. Según cómo se acceda a la información, pueden distinguirse soportes de **acceso directo**, que permiten localizar datos sin recorrer todo el contenido anterior, y soportes de **acceso secuencial**, donde la información se procesa siguiendo un orden.

Los sistemas de almacenamiento han evolucionado desde las **cintas magnéticas** y los primeros discos hasta dispositivos actuales con mayor capacidad y velocidad.

Entre los soportes más utilizados se encuentran:

Soporte	Características principales	Uso habitual
HDD	Almacenamiento magnético, gran capacidad y coste reducido.	Equipos, servidores y almacenamiento masivo.
SSD	Utiliza memoria flash, sin partes móviles y con **acceso rápido**.	Equipos personales y servidores.
SSD NVMe	SSD conectado mediante **PCIe**, con mayor rendimiento que las unidades SATA.	Sistemas que requieren alta velocidad de acceso.
Cinta magnética	Gran capacidad y bajo coste por unidad almacenada, pero con **acceso secuencial**.	Copias de seguridad y archivo a largo plazo.
Soportes ópticos	CD, DVD o Blu-ray. Actualmente tienen un uso mucho más limitado.	Distribución o conservación puntual de información.



Elaboración propia. *Soportes de información.* (CC BY-NC-SA)

Según la forma de acceder a los datos se distinguen:

- **Acceso directo:** permite localizar una posición concreta sin recorrer todos los datos

anteriores. Es habitual en **HDD y SSD**.

- **Acceso secuencial:** los datos se recorren siguiendo su orden de almacenamiento hasta alcanzar el dato buscado. Es característico de las cintas magnéticas.

El **almacenamiento en red** y el **almacenamiento en la nube** permiten acceder a datos alojados en sistemas remotos, aunque físicamente la información sigue almacenándose en dispositivos como discos o unidades de estado sólido.

Para saber más

Consulta la evolución de los soportes de almacenamiento, desde los primeros medios físicos hasta los sistemas magnéticos, ópticos y de estado sólido.

Evolución de los soportes de almacenamiento

Actividad desplegable

Una empresa necesita elegir el **soporte de almacenamiento** más adecuado para tres necesidades diferentes:

1. El servidor principal debe ejecutar una base de datos con muchas consultas y necesita gran velocidad de acceso.
2. Las copias de seguridad mensuales deben conservarse durante varios años y se consultarán muy pocas veces.
3. Un equipo de oficina necesita almacenar una gran cantidad de archivos con un coste reducido por capacidad.

Relaciona cada situación con el soporte más adecuado:

Situación	Soporte de información
1. Base de datos con muchas consultas y necesidad de alta velocidad.	
2. Copias de seguridad de larga duración y acceso poco frecuente.	
3. Gran capacidad de almacenamiento con prioridad en el coste.	

Comprobar Mostrar retroalimentación

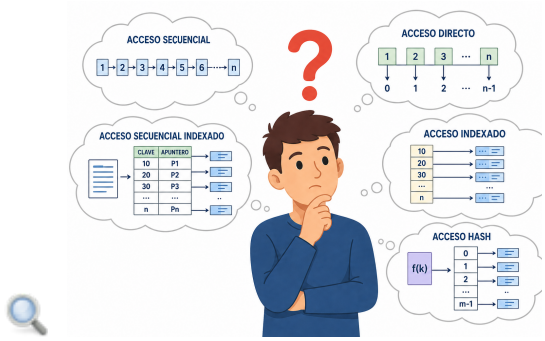
1. **SSD NVMe:** es la opción más adecuada cuando se necesita **alta velocidad de acceso** y bajo tiempo de respuesta.
2. **Cinta magnética:** resulta apropiada para **copias de seguridad y archivo a largo plazo**, donde importa más la capacidad y el coste que la rapidez de acceso.
3. **HDD:** ofrece una buena relación entre **capacidad y coste**, por lo que sigue siendo útil para almacenar grandes cantidades de información cuando no se necesita el máximo rendimiento.

2.3.- Métodos de acceso.

Los **métodos de acceso a ficheros** determinan cómo se organizan y localizan los registros almacenados. Su elección influye directamente en la **velocidad de acceso**, la facilidad para actualizar los datos y el aprovechamiento del almacenamiento.

Los principales objetivos de los métodos de acceso son:

- Conseguir un **acceso rápido** a los registros.
- Facilitar la **consulta y actualización** de la información.
- Aprovechar mejor el **espacio de almacenamiento**.
- Adaptar la organización del fichero a la forma en que se van a utilizar los datos.



Elaboración propia. *Métodos de acceso.* (CC BY-NC-SA)

Los métodos más habituales que se estudiarán son:

Método	Características	Ejemplo
Secuencial	Los registros se recorren siguiendo un **orden**, normalmente desde el principio hasta localizar el dato.	Leer las líneas de un fichero de registro una tras otra.
Directo o aleatorio	Permite acceder a una **posición concreta** sin recorrer previamente todos los registros.	Consultar un registro situado en una posición conocida del fichero.
Indexado	Utiliza un **índice** que relaciona las claves con la posición de los registros.	Buscar un cliente mediante su identificador utilizando un índice.
Secuencial indexado	Combina un **índice** con el recorrido secuencial de una parte del fichero.	Localizar primero un bloque mediante un índice y recorrer después sus registros.
Calculado o hash	Obtiene la posición aplicando una **función hash** sobre una clave.	Calcular la ubicación de un registro a partir de su identificador.

La elección del método dependerá del tipo de operaciones que se realicen con mayor frecuencia y de la necesidad de priorizar **rapidez, actualización o recorrido completo de los datos**.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. En un acceso secuencial, para localizar un registro puede ser necesario recorrer previamente otros registros.

○ Verdadero ○ Falso

Verdadero

Este método sigue un **orden de lectura**, por lo que no permite saltar directamente a cualquier registro.

2. En un acceso indexado, la posición del registro se obtiene siempre aplicando una función matemática sobre su clave.

○ Verdadero ○ Falso

Falso

El acceso indexado utiliza un **índice** que relaciona las claves con la posición de los registros. La aplicación de una función corresponde al acceso **calculado o hash**.

3. El acceso directo o aleatorio permite localizar una posición concreta sin recorrer necesariamente todos los registros anteriores.

○ Verdadero ○ Falso

Verdadero

Su principal característica es permitir un **acceso directo** a una ubicación determinada.

4. El método hash necesita consultar siempre una tabla de índices antes de acceder al registro.

○ Verdadero ○ Falso

Falso

En el acceso hash, la posición se obtiene mediante una **función de transformación** aplicada a la clave.

2.4.- Ficheros secuenciales.

Un **fichero secuencial** almacena sus registros siguiendo un orden y se procesa normalmente **desde el principio hacia el final**. Para localizar un registro puede ser necesario leer los anteriores, por lo que resulta adecuado cuando interesa recorrer gran parte o la totalidad del fichero.

Los registros se procesan **uno tras otro** y el final del fichero suele detectarse mediante **EOF (End Of File)**, que indica que ya no quedan más datos por leer.

Los registros pueden disponer de un **campo clave** que permita identificarlos. Si el fichero está ordenado por esa clave, algunas operaciones secuenciales pueden realizarse con mayor facilidad.

Sus principales características son:

- La lectura se realiza normalmente **hacia delante**.
- Los registros siguen una **estructura definida**.
- Puede almacenarse tanto en soportes de **acceso secuencial** como de acceso directo.
- Aprovecha bien el espacio al almacenar los registros de forma consecutiva.
- Insertar un registro entre otros ya existentes puede requerir **reescribir o reorganizar** parte del fichero.
- Resulta especialmente útil cuando se necesita **recorrer muchos registros o el fichero completo**.

Ministerio de Educación. *Fichero secuencial.* (CC BY-NC-SA)

Autoevaluación

Una aplicación genera diariamente un fichero con miles de operaciones ordenadas cronológicamente. Al finalizar la jornada, otro proceso debe leer **todas las operaciones** para elaborar un informe.

¿Qué característica hace especialmente adecuada una organización secuencial?

- a) Permite localizar cualquier registro directamente mediante una dirección calculada.
- b) Utiliza necesariamente un índice para acceder a cada registro.
- c) Resulta eficiente cuando los datos deben recorrerse de forma completa y ordenada.
- d) Permite insertar registros en cualquier posición sin reorganizar el fichero.

Incorrecta. Ese funcionamiento corresponde a métodos como el **acceso calculado o hash**.

Incorrecta. Un fichero secuencial puede recorrerse sin utilizar ningún **índice**.

Correcta. El acceso secuencial resulta adecuado cuando es necesario **procesar el fichero completo siguiendo su orden**.

Incorrecta. Insertar registros en posiciones intermedias puede requerir **reorganizar o reescribir** parte del fichero.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

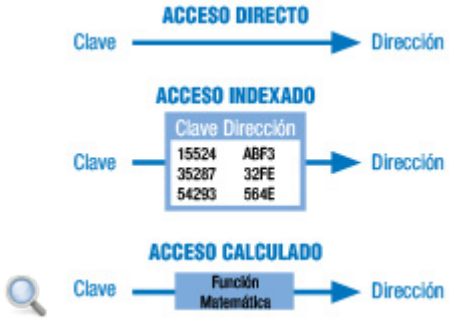
2.5.- Ficheros de acceso directo.

Los **ficheros de acceso directo o aleatorio** permiten localizar un registro sin recorrer previamente todos los anteriores. El acceso se realiza mediante una **posición** o una **clave**, por lo que resultan adecuados cuando se necesita recuperar o modificar información concreta con rapidez.

Cada registro se almacena en una **posición determinada** dentro de un soporte direccionable. Para localizarlo se utiliza una dirección o mecanismo equivalente, evitando recorrer el fichero desde el principio.

Campo clave: campo que permite **identificar y localizar** un registro dentro del fichero.

En el acceso directo, la clave puede estar relacionada directamente con la posición del registro. Otros métodos, como el **indexado** o el **calculado mediante hash**, utilizan mecanismos adicionales para obtener esa ubicación.



Ministerio de Educación . *Obtención de direcciones físicas a partir de la clave según el modo de acceso.* (CC BY-NC-SA)

Sus características principales son:

- Permite un **posicionamiento rápido** sobre un registro concreto.
- Suele trabajar con registros de **longitud fija** para facilitar el cálculo de posiciones.
- Permite realizar operaciones de **lectura y escritura** sobre registros concretos.
- La actualización puede realizarse sobre el mismo fichero, sin tener que recorrerlo completamente.
- Requiere un soporte que permita **acceso directo** a las posiciones almacenadas.
- Es adecuado cuando se conoce la **clave o posición** del registro que se quiere consultar.

Ejemplo:

Si cada registro ocupa 100 bytes y queremos acceder al registro número 50, el sistema puede calcular su posición aproximada dentro del fichero sin leer previamente los 49 registros anteriores.

Actividad desplegable

Completa las siguientes frases con los conceptos adecuados:

1. En un fichero de acceso directo no es necesario recorrer todos los registros [ ] para localizar uno concreto.
2. El campo utilizado para identificar y localizar un registro se denomina [ ] .
3. Este tipo de organización suele utilizar registros de longitud [ ] para facilitar el posicionamiento.
4. Para utilizar acceso directo, el soporte debe permitir acceder a posiciones de forma [ ] .

Comprobar | Mostrar retroalimentación

1. **Anteriores:** Correcto. En el acceso directo no es necesario recorrer los **registros anteriores** para llegar al registro buscado.
2. **Campo clave:** Correcto. El **campo clave** permite identificar y localizar un registro dentro del fichero.
3. **Fija:** Correcto. Los registros de **longitud fija** facilitan el cálculo de la posición que ocupa cada registro.
4. **Directa:** Correcto. Este tipo de organización necesita un soporte que permita acceder de forma **directa** a posiciones concretas.

2.6.- Ficheros indexados.

Los **ficheros indexados** utilizan una estructura auxiliar llamada **índice** para localizar los registros sin tener que recorrer todo el fichero. El índice relaciona una **clave** con la posición donde se encuentra cada registro, permitiendo combinar **acceso directo** y **acceso secuencial**.

Un fichero indexado se divide en dos zonas:

1. **Zona de registros:** contiene los datos almacenados.
2. **Zona de índices:** contiene las **claves** y las posiciones asociadas a los registros.

Cuando se busca un registro, primero se consulta el **índice** para localizar su posición y después se accede al registro correspondiente.

Ministerio de Educacion. *Organización de ficheros indexados.* (CC BY-NC-SA)

Las características más importantes son:

- El fichero debe disponer de un **campo clave** que permita identificar los registros.
- Una **clave primaria** identifica cada registro de forma única y no admite duplicados.
- Pueden existir **claves alternativas** adicionales.
- Permite realizar **acceso directo**, utilizando el índice para localizar un registro.
- También permite **acceso secuencial**, recorriendo los registros según el orden de la clave.
- El **orden lógico** determinado por el índice no tiene por qué coincidir con el **orden físico** en el que se almacenaron los registros.
- El índice ocupa **espacio adicional** y debe mantenerse actualizado cuando cambian los datos.

Ejemplo:

En un fichero de clientes, un índice sobre el **DNI** puede indicar directamente dónde se encuentra almacenado cada registro, evitando recorrer todos los clientes anteriores.

Actividad desplegable

Relaciona cada concepto con su descripción correcta.

Concepto	Descripción
	Estructura que relaciona una **clave** con la posición de un registro.
	Identifica de forma **única** cada registro y no admite duplicados.
	Campo adicional que puede utilizarse para realizar búsquedas.
	Orden determinado por el **índice**, independientemente de la posición física de los registros.

Comprobar Mostrar retroalimentación

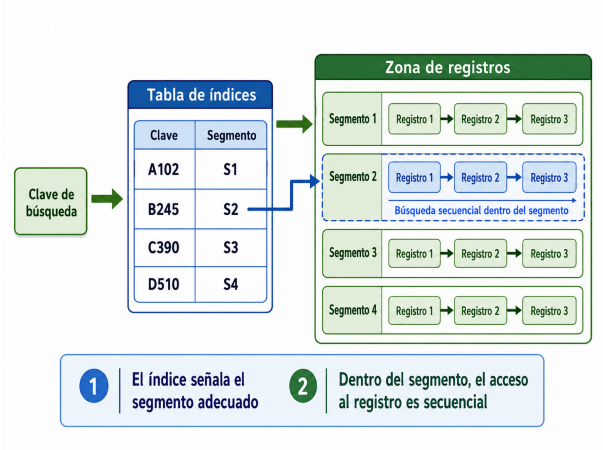
1. **Índice:** permite localizar rápidamente un registro mediante la relación entre su **clave y dirección**.
2. **Clave primaria:** garantiza que cada registro pueda identificarse de forma **única**.
3. **Clave alternativa:** proporciona otra forma de localizar registros sin sustituir a la clave primaria.
4. **Orden lógico:** depende del **índice** y puede ser diferente del orden físico en el que se almacenaron los datos.

2.7.- Otros (secuenciales indexados, hash.).

Ficheros secuenciales indexados

También se denominan **parcialmente indexados**. Disponen de una **zona de índices** y una **zona de registros**, pero esta última se divide en **segmentos ordenados**.

El índice permite localizar el segmento adecuado y, una vez dentro de él, el registro se busca de forma **secuencial**.



Elaboración propia. *Ficheros secuenciales indexados.* (CC BY-NC-SA)

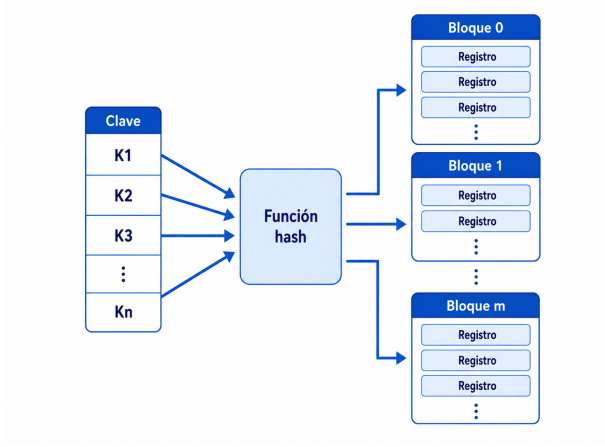
Sus principales características son:

- Permiten **acceso secuencial** y acceso mediante índice.
- Los registros pueden leerse ordenados por el **campo clave**.
- Ocupan más espacio que un fichero puramente secuencial por la existencia del **índice**.
- Resultan útiles cuando se combinan búsquedas concretas con recorridos amplios del fichero.

Ejemplo:

Un fichero de clientes puede dividirse en segmentos según rangos de identificadores. El índice localiza primero el segmento correspondiente y después se recorre ese bloque hasta encontrar el cliente.

Ficheros de acceso calculado o hash



Elaboración propia. *Ficheros de acceso calculado o hash* (CC BY-NC-SA)

En el acceso **hash**, la posición del registro se obtiene aplicando una **función hash** sobre su clave, sin necesidad de consultar previamente una tabla de índices.

Ejemplo:

Si la clave de un registro es 1357, una función puede transformar ese valor para determinar directamente la posición donde debe almacenarse o buscarse.

El principal problema aparece cuando dos claves diferentes generan la misma posición. Esta situación se denomina **colisión**.

Para resolver las colisiones pueden utilizarse distintas técnicas, *como:*

1. **Zona de desbordamiento** para almacenar registros que no pueden ocupar su posición calculada.
2. Técnicas de **resolución de colisiones** que buscan otra posición disponible.

Una buena función hash debe distribuir las claves de forma **uniforme** y reducir al mínimo el número de colisiones.

*El método tradicional de **módulo** calcula la posición mediante el resto de una división:*

dirección = clave mod número de posiciones

Autoevaluación

En un fichero hash se aplica una función a las claves de varios registros y dos claves diferentes generan la misma posición.

¿Qué situación se ha producido?

- a) Una indexación.
- b) Una fragmentación.
- c) Una colisión.
- d) Un acceso secuencial.

Incorrecta. La indexación utiliza una **estructura auxiliar** que relaciona claves con posiciones.

Incorrecta. La fragmentación consiste en dividir información en partes y no describe este problema.

Correcta. Una **colisión** ocurre cuando dos claves diferentes producen el mismo resultado al aplicar la función hash.

Incorrecta. El acceso secuencial implica recorrer registros siguiendo un orden, no calcular su posición mediante una función.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

2.8.- Parámetros de utilización.

Los **parámetros de utilización** permiten analizar cómo se usa un fichero y ayudan a elegir la organización más adecuada. Los principales son su **capacidad**, la **frecuencia** con la que se consulta o modifica, su **volatilidad** y el ritmo al que **crece**.

La organización de un fichero debe adaptarse al **uso real de sus datos**. No tendrá las mismas necesidades un archivo que apenas cambia que otro en el que se realizan continuamente consultas, inserciones y borrados.

Parámetro	Qué mide	Indicadores	Ejemplo
Capacidad o volumen	Cantidad de **información almacenada** en el fichero.	Número de registros y tamaño medio de cada registro.	Un fichero con **50 000 clientes** ocupa más espacio que otro con 5 000.
Actividad	Cantidad de **consultas y modificaciones** realizadas.	**Tasa** y **frecuencia de consulta o modificación**.	Un catálogo de productos consultado y actualizado continuamente tiene **actividad alta**.
Volatilidad	Cantidad de **inserciones y borrados** que experimenta el fichero.	**Tasa** y **frecuencia de renovación**.	Un fichero de usuarios con muchas **altas y bajas** presenta volatilidad elevada.
Crecimiento	Variación del **tamaño del fichero** a lo largo del tiempo.	**Tasa de crecimiento**.	Un fichero de pedidos que incorpora miles de registros cada mes tiene **crecimiento continuo**.

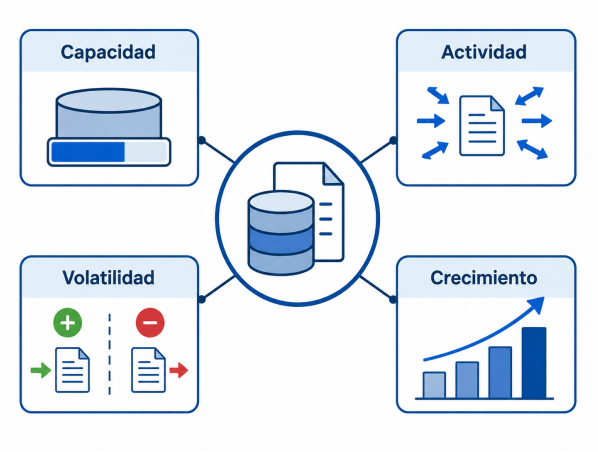
*La **capacidad** puede estimarse a partir del número de registros y del tamaño medio de cada uno:*

Capacidad aproximada = número de registros × tamaño medio del registro

*Dentro de la **actividad** y la **volatilidad** se distingue entre:*

- **Tasa:** proporción de registros afectados en cada tratamiento.
- **Frecuencia:** número de veces que se realiza ese tratamiento durante un periodo determinado.

Así, un fichero puede presentar una **actividad alta** porque se consulta constantemente y, al mismo tiempo, una **volatilidad baja** si apenas se insertan o eliminan registros.



Elaboración propia. *Parámetros de utilización.* (CC BY-NC-SA)

Autoevaluación

La volatilidad de un fichero es un parámetro que indica:

- La variación del volumen del fichero.
- La cantidad de veces que se abre o cierra el fichero.
- El peso de los procesos de inserción y borrado en dicho fichero (frecuencia de renovación).

No es correcto. Eso sería el crecimiento, no la volatilidad.

Incorrecto. Eso sería la actividad, no la volatilidad.

Efectivamente, mide la cantidad de inserciones y borrados que se efectúan en el fichero.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

3.- Bases de datos.

Caso práctico

Ada, **Juan** y **María** deben decidir cómo gestionar la información de las aplicaciones de BK Programación.

Stockbyte (CC BY-NC-SA)

—Podríamos seguir utilizando **ficheros**, pero si varias aplicaciones necesitan compartir los mismos datos aparecerán problemas —comenta **Juan**.

—Exacto —responde **Ada**—. Una **base de datos** permite centralizar la información, reducir duplicidades y facilitar que distintas aplicaciones trabajen con los mismos datos de forma controlada.

María añade que elegir correctamente cómo almacenar la información es fundamental para garantizar su **coherencia, seguridad y disponibilidad**.

Una **base de datos** es un conjunto organizado de **datos relacionados** que pueden ser **almacenados**, **consultados** y **modificados** de forma controlada. Su finalidad es **facilitar** el **acceso** a la información, reducir problemas como la **redundancia** y mantener los datos **coherentes, seguros y disponibles** para diferentes usuarios y aplicaciones.

Los sistemas basados únicamente en ficheros presentan limitaciones cuando aumenta la cantidad de información o intervienen varias aplicaciones.

Entre los problemas más habituales se encuentran:

- **Duplicidad de datos:** la misma información puede almacenarse varias veces.
- **Inconsistencia:** distintas copias de un mismo dato pueden contener valores diferentes.
- **Dependencia entre programas y ficheros:** una modificación en la estructura del fichero puede obligar a modificar las aplicaciones.
- **Acceso compartido limitado:** resulta más complejo coordinar varios usuarios o programas trabajando simultáneamente.
- **Seguridad y control:** gestionar permisos y proteger la información resulta más difícil.

Las bases de datos permiten mantener la información en una estructura común y

gestionar su acceso mediante un **Sistema Gestor de Bases de Datos (SGBD)**, facilitando el trabajo simultáneo de aplicaciones y usuarios.

Ejemplo:

En una tienda online, los datos de **clientes, productos, pedidos y pagos** pueden estar relacionados dentro de una misma base de datos. Así, diferentes partes de la aplicación utilizan información común sin mantener copias independientes.

macrovector . *Bases de datos.* (CC BY-NC-SA)

Reflexiona

Una empresa mantiene los datos de sus clientes en tres aplicaciones diferentes. Cada una dispone de su propio fichero y, cuando un cliente cambia de dirección, no siempre se actualizan los tres.

¿Qué problema se está produciendo y qué ventaja aportaría utilizar una base de datos común?

Mostrar retroalimentación

Existe **redundancia**, porque la misma información aparece almacenada varias veces, y puede producirse **inconsistencia** si las copias no se actualizan de la misma forma. Una **base de datos común** permitiría que las aplicaciones trabajasen sobre una información compartida, reduciendo duplicidades y facilitando que los datos permanezcan **coherentes y actualizados**.

3.1.- Conceptos.

Una **base de datos** es un conjunto organizado de **datos relacionados** que representa información de una realidad concreta. Además de los propios datos, incluye información sobre su estructura, lo que permite gestionarlos con menor **redundancia**, mayor **coherencia** y cierta independencia respecto a las aplicaciones que los utilizan.

Una base de datos no contiene solo datos, sino también información sobre cómo están definidos y organizados. Esa información recibe el nombre de **metadatos** y se almacena en el **diccionario de datos** o **catálogo**.

Los conceptos básicos asociados a una base de datos son:

1. **Entidad:** objeto, persona, lugar o concepto sobre el que se almacena información.
   - **Ejemplo:** cliente, producto, pedido o empleado.
2. **Atributo:** característica que describe una entidad.
   - **Ejemplo:** nombre, correo electrónico o fecha de alta de un cliente.
3. **Registro:** conjunto de datos que describe una ocurrencia concreta de una entidad.
   - **Ejemplo:** los datos completos de un cliente determinado.
4. **Campo:** valor individual almacenado dentro de un registro.
   - **Ejemplo:** el correo electrónico de ese cliente.
5. **Metadato:** información que describe la **estructura de los datos**, como nombres de campos, tipos o restricciones.

Magnific. *Conceptos.* (CC BY-NC-SA)

Ventajas principales

- **Acceso múltiple:** varios usuarios o aplicaciones pueden trabajar con los mismos datos.
- **Reducción de redundancia:** se evita repetir información innecesariamente.
- **Integridad:** ayuda a mantener los datos **válidos y coherentes**.
- **Seguridad:** permite controlar qué usuarios pueden acceder o modificar la información.
- **Recuperación ante fallos:** facilita restaurar los datos tras determinados errores.
- **Independencia lógica y física:** los cambios en la forma de almacenar o estructurar los datos pueden reducir el impacto sobre las aplicaciones.
- **Consulta directa:** permite recuperar información mediante lenguajes de consulta, como **SQL**.

Actividad desplegable

Relaciona cada concepto con su definición correcta.

Concepto	Definición
	Información que describe la estructura de los datos.
	Conjunto de datos de una ocurrencia concreta de una entidad.
	Objeto, persona o concepto sobre el que se almacena información.
	Característica que describe una entidad.

Comprobar Mostrar retroalimentación

1. **Entidad** → *Objeto, persona, lugar o concepto sobre el que se almacena información.*
2. **Atributo** → *Característica que describe una entidad.*
3. **Registro** → *Conjunto de datos que describe una ocurrencia concreta de una entidad.*
4. **Metadato** → *Información que describe la estructura de los datos, como nombres de campos, tipos o restricciones.*

3.2.- Usos.

Las **bases de datos** son utilizadas por distintos perfiles, desde quienes las diseñan y administran hasta las aplicaciones y usuarios que consultan su información. Se emplean en cualquier sistema que necesite **almacenar, relacionar, consultar y actualizar datos** de forma organizada.

¿Quién utiliza las bases de datos?

Perfil	Funciones principales
Administrador de bases de datos (DBA)	Gestiona el **SGBD**, usuarios y permisos, seguridad, copias de seguridad, rendimiento y disponibilidad de los datos.
Diseñador de bases de datos	Define la **estructura de la información**, las entidades, relaciones y restricciones necesarias para representar correctamente los datos.
Desarrollador de aplicaciones	Crea aplicaciones que **consultan y modifican** los datos mediante lenguajes y tecnologías de acceso a bases de datos.
Usuario final	Utiliza aplicaciones para consultar o introducir información sin necesitar conocer cómo está organizada internamente la base de datos.

En proyectos actuales estos perfiles pueden **solaparse**, especialmente en equipos pequeños, donde una misma persona puede diseñar la base de datos, desarrollar la aplicación y realizar determinadas tareas de administración.

¿Para qué se utilizan?

Las bases de datos aparecen en prácticamente todos los sectores que necesitan gestionar información:

Ámbito	Datos gestionados
Comercio electrónico	Clientes, productos, stock, pedidos y pagos.
Banca	Cuentas, movimientos, préstamos y transacciones.
Sanidad	Pacientes, citas, historiales y tratamientos.

Ámbito	Datos gestionados
Educación	Alumnado, matrículas, cursos y calificaciones.
Administraciones públicas	Ciudadanía, expedientes, registros y trámites.
Transporte y reservas	Usuarios, horarios, disponibilidad y reservas.
Aplicaciones web	Usuarios, permisos, contenidos, sesiones y actividad.

La importancia de una base de datos no depende únicamente de la cantidad de información almacenada, sino de la necesidad de mantener los datos **relacionados, disponibles y actualizados** para diferentes procesos y usuarios.

Autoevaluación

Una empresa desarrolla una aplicación web de reservas. Una persona define las tablas y relaciones, otra configura permisos y copias de seguridad, y los clientes realizan reservas desde la aplicación.

¿Qué asociaciones entre perfiles y funciones son correctas? Puede haber más de una respuesta

- [ ] a) El diseñador de bases de datos define cómo se estructuran y relacionan los datos.
- [ ] b) El usuario final debe administrar las copias de seguridad del SGBD.
- [ ] c) El DBA puede gestionar permisos, seguridad y recuperación de la información.
- [ ] d) El desarrollador de aplicaciones crea el software que consulta y modifica los datos.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

3.3.- Ubicación de la información.

La información de una **base de datos** puede almacenarse en un único equipo, en varios dispositivos o en sistemas de almacenamiento conectados por **red**. La elección depende del **volumen de datos**, el **rendimiento**, la **disponibilidad** y la forma en que deben acceder **usuarios y aplicaciones**.

Las bases de datos actuales suelen almacenarse sobre **HDD** o **SSD**, ya sea directamente en un servidor o mediante infraestructuras de **almacenamiento compartido**.

Principales sistemas de almacenamiento

Sistema	Características	Ejemplo de uso
Almacenamiento local	Los datos se almacenan en **discos o SSD** conectados directamente al servidor.	Base de datos instalada en un servidor con **SSD NVMe**.
RAID	Combina varios discos para mejorar el **rendimiento**, la **disponibilidad** o la **tolerancia a fallos**, según el nivel utilizado.	Servidor con **RAID 1** para mantener una copia redundante de los datos.
NAS	Sistema conectado a la **red** que comparte información principalmente a nivel de **archivos**.	Copias de seguridad o archivos compartidos entre varios servidores.
SAN	Red especializada que proporciona almacenamiento a nivel de **bloques** a distintos servidores.	Centro de datos con varios servidores conectados a una **cabina de almacenamiento**.
Almacenamiento en la nube	Recursos almacenados en una **infraestructura remota** y accesibles a través de la red.	Base de datos desplegada sobre una plataforma **cloud**.

Elaboración propia. *Ubicación de la información.* (CC BY-NC-SA)

Pregunta Verdadero-Falso

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. Un sistema RAID puede combinar varios discos para mejorar la redundancia, el rendimiento o ambos.

○ Verdadero ○ Falso

Verdadero

RAID organiza varios dispositivos de almacenamiento según un nivel concreto para aportar **tolerancia a fallos**, rendimiento o ambas características.

2. Un sistema NAS proporciona normalmente almacenamiento a nivel de bloques, igual que una SAN.

○ Verdadero ○ Falso

Falso

Un **NAS** trabaja principalmente a nivel de **archivos**, mientras que una **SAN** proporciona acceso a bloques de almacenamiento.

3. Una SAN puede ofrecer almacenamiento a varios servidores mediante una red especializada.

○ Verdadero ○ Falso

Verdadero

Una SAN permite que varios servidores accedan a almacenamiento compartido mediante una red especializada y normalmente trabaja a nivel de bloques.

4. El almacenamiento en la nube implica que los datos ya no se guardan en dispositivos físicos.

○ Verdadero ○ Falso

Falso

Falso. Los datos siguen almacenándose en **servidores, discos o SSD** dentro de la infraestructura del proveedor.

4.- Modelos de bases de datos.

Caso práctico

Christopher Walsh, Harvard Medical School . *Evolución.* (CC BY-NC-SA)

Juan, María y **Ada** están preparando el diseño de una nueva base de datos para una aplicación.

—Tenemos claros los datos que necesitamos, pero ¿cómo decidimos la forma de **organizarlos y relacionarlos**? —pregunta **Juan**.

—Para eso existen los **modelos de bases de datos** —responde **María**—. Cada modelo establece una manera diferente de representar los datos, sus relaciones y las operaciones que pueden realizarse sobre ellos.

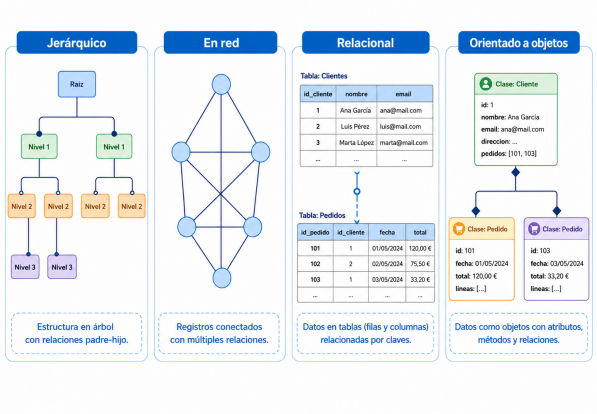
Ada señala que elegir un modelo adecuado facilita el **diseño**, la consulta de la información y su mantenimiento.

Un **modelo de base de datos** define cómo se **estructuran, relacionan y representan los datos** dentro de una base de datos. Cada modelo utiliza unas estructuras y reglas determinadas para describir la información y establecer las relaciones existentes entre sus elementos.

Los principales modelos que han marcado la evolución de las bases de datos son:

- **Modelo jerárquico:** organiza los datos mediante una estructura de **árbol**, con relaciones padre-hijo.
- **Modelo en red:** permite que un registro pueda estar relacionado con **varios registros** diferentes.
- **Modelo relacional:** organiza la información en **tablas relacionadas** y es el modelo más extendido en bases de datos de propósito general.
- **Modelo orientado a objetos:** representa la información mediante **objetos**, incorporando conceptos propios de la programación orientada a objetos.
- **Otros modelos:** incluyen enfoques como los modelos **documentales, clave-valor, columnares o de grafos**, utilizados especialmente en determinados sistemas NoSQL.

La elección del modelo depende de la **estructura de los datos**, las relaciones existentes entre ellos y las necesidades de la aplicación.



Elaboración propia. *Modelos de bases de datos.* (CC BY-NC-SA)

Reflexiona

Una empresa necesita almacenar información sobre usuarios y las personas con las que interactúan en una red social. Cada usuario puede estar relacionado con muchos otros usuarios y esas relaciones pueden cambiar continuamente.

¿Crees que todos los modelos de bases de datos representarían estas relaciones con la misma facilidad? ¿Qué debería tenerse en cuenta al elegir el modelo?

Mostrar retroalimentación

No todos los modelos representan las relaciones con la misma facilidad. La elección debe considerar la **estructura de la información**, la cantidad y tipo de **relaciones** y las operaciones que se realizarán con mayor frecuencia. Por ejemplo, cuando las relaciones entre elementos tienen especial importancia, pueden utilizarse modelos diseñados específicamente para representar conexiones complejas, como los **modelos de grafos**.

4.1.- Modelo jerárquico.

El **modelo jerárquico** organiza los datos mediante una estructura en forma de **árbol**, formada por distintos niveles. Cada elemento puede tener **un único padre** y varios hijos, por lo que las relaciones siguen principalmente un esquema **uno a muchos (1:N)**.

Este modelo se popularizó con **IMS (Information Management System)** de IBM y fue uno de los primeros modelos utilizados ampliamente para gestionar grandes volúmenes de información.

*Los datos se organizan mediante **nodos o segmentos** relacionados entre sí:*

1. **Nodo raíz:** elemento situado en el nivel superior, sin padre.
2. **Nodo padre:** puede tener uno o varios **nodos hijos**.
3. **Nodo hijo:** depende de un único padre, aunque puede tener sus propios descendientes.
4. **Segmento:** estructura lógica utilizada para almacenar los datos dentro de la jerarquía.

Ejemplo:

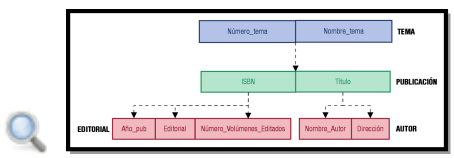
Una empresa puede organizar su información como:

Empresa → Departamentos → Empleados

Una empresa puede tener varios **departamentos** y cada departamento varios **empleados**, pero cada empleado quedaría vinculado a un único departamento dentro de esa jerarquía.

Su principal ventaja es la **simplicidad y rapidez** cuando los datos presentan una estructura claramente jerárquica. Sin embargo, resulta poco flexible cuando un elemento necesita relacionarse con **varios padres** o existen relaciones complejas.

Actualmente no es el modelo predominante frente al **modelo relacional** y otros modelos modernos, aunque sigue presente en **sistemas heredados y entornos especializados**, como determinadas instalaciones de IBM IMS.



Elaboración propia. *Modelo de datos jerárquico* (CC BY-NC-SA)

El modelo jerárquico organiza los datos en **niveles**, donde cada nodo hijo depende de un **único padre** y un padre puede tener varios hijos.

Actividad desplegable

Completa las siguientes afirmaciones con el término adecuado.

1. El modelo jerárquico representa la información mediante una estructura en forma de [ ] .
2. Cada nodo puede tener un único [ ] , aunque puede disponer de varios hijos.
3. La relación característica del modelo jerárquico es [ ] .
4. El elemento situado en el nivel superior de la estructura recibe el nombre de nodo [ ] .

Comprobar | Mostrar retroalimentación

1. **Árbol:** la información se distribuye en **niveles jerárquicos** conectados entre sí.
2. **Padre:** una de las principales limitaciones del modelo es que cada hijo depende de **un único nodo superior**.
3. **Uno a muchos (1:N):** un padre puede relacionarse con **varios hijos**, pero cada hijo tiene un solo padre.
4. **Raíz:** es el nodo situado en la **parte superior de la jerarquía** y no depende de ningún otro.

4.2.- Modelo en red.

El **modelo en red** organiza la información mediante **registros o nodos** conectados por **enlaces**, permitiendo representar relaciones más complejas que el modelo jerárquico. A diferencia de este, un registro puede estar relacionado con **varios registros superiores**, por lo que admite estructuras **muchos a muchos (N:M)**.

El **modelo en red** surgió en los años sesenta como respuesta a las limitaciones del modelo jerárquico. Fue impulsado por **CODASYL y se utilizó en sistemas como IDMS.**

Sus elementos principales son:

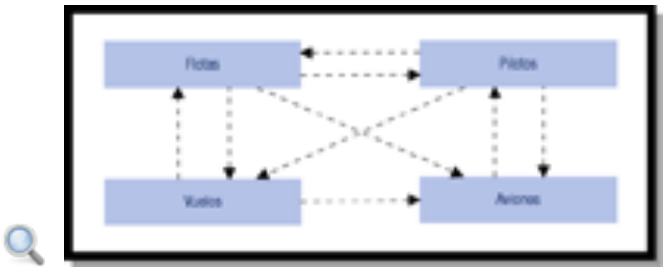
- **Registros o nodos:** almacenan la información.
- **Enlaces:** representan las relaciones entre los registros.
- Un registro puede estar conectado con **varios registros diferentes**, sin quedar limitado a un único padre.

Ejemplo:

En una compañía aérea, un **piloto** puede participar en varios vuelos y un **vuelo** puede estar relacionado con distintos pilotos, aviones y rutas.

Este modelo permite representar relaciones complejas con menos duplicación que el modelo jerárquico, pero su estructura resulta **más difícil de diseñar, consultar y mantener**.

Actualmente tiene sobre todo interés **histórico** y permanece en algunos **sistemas heredados**, mientras que para la mayoría de aplicaciones se utilizan modelos más flexibles y lenguajes de consulta de mayor nivel.



Elaboración propia. *Ejemplo de Modelo de datos en red* (CC BY-NC-SA)

Autoevaluación

¿Cuáles de las siguientes características corresponden al modelo en

red? Puede haber más de una respuesta correcta.

- [ ] a) Un registro puede estar relacionado con varios registros diferentes.
- [ ] b) Cada nodo debe depender obligatoriamente de un único padre.
- [ ] c) Permite representar relaciones muchos a muchos.
- [ ] d) Su estructura suele resultar más compleja de gestionar que la del modelo jerárquico.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

4.3.- Modelo relacional.

El **modelo relacional**, propuesto por **Edgar F. Codd en 1970**, representa los datos mediante **tablas relacionadas entre sí**. Cada tabla almacena información sobre un tipo de elemento y organiza los datos en **filas y columnas**, utilizando claves para identificar registros y establecer relaciones.

Actualmente, el modelo relacional sigue siendo uno de los más utilizados en sistemas de gestión de bases de datos.

Sus conceptos fundamentales son:

- **Relación o tabla:** estructura formada por filas y columnas.
- **Tupla o fila:** representa una ocurrencia concreta de los datos almacenados.
- **Atributo o columna:** característica que describe los elementos de la tabla.
- **Dominio:** conjunto de **valores válidos** que puede tomar un atributo.
- **Clave:** atributo o conjunto de atributos utilizado para **identificar registros** o relacionar tablas.

Ejemplo:

En una tabla **ALUMNOS**, cada fila representa un alumno y columnas como **id_alumno, nombre, apellidos o fecha_nacimiento** representan sus atributos.

En el modelo relacional:

- Cada tabla tiene un **nombre identificativo**.
- Cada columna representa un **atributo determinado**.
- Los valores de una columna pertenecen a un mismo **dominio**.
- Las filas no necesitan mantener un **orden físico determinado**.
- Una **clave primaria** permite identificar de forma única cada fila.
- Las relaciones entre tablas pueden establecerse mediante **claves**.

El lenguaje más utilizado para trabajar con bases de datos relacionales es **SQL (Structured Query Language)**, que permite **consultar, insertar, modificar y eliminar datos**, además de definir y administrar estructuras de la base de datos.

Normalización

La **normalización** es un proceso de diseño que organiza las tablas y sus relaciones para reducir **redundancias** y evitar problemas de **inserción, modificación o eliminación** de datos.

Elaboración propia. *Relación en el Modelo Relacional* (CC BY-NC-SA)

Actividad desplegable

Completa cada situación seleccionando el concepto adecuado.

1. En una tabla CLIENTES, cada cliente almacenado constituye una [ ] .
2. La columna **correo_electronico** constituye un [ ] .
3. El conjunto de valores admitidos por una columna pertenece a su [ ] .
4. El campo que permite distinguir de forma única a cada cliente puede actuar como [ ] .

Comprobar | Mostrar retroalimentación

1. **Tupla:** representa una **fila concreta** de una relación y contiene los valores correspondientes a un elemento almacenado.
2. **Atributo:** representa una **característica** de los datos y se corresponde con una columna de la tabla.
3. **Dominio:** establece el conjunto o tipo de **valores permitidos** para un atributo.
4. **Clave primaria:** permite **identificar de forma única** cada fila de una tabla.

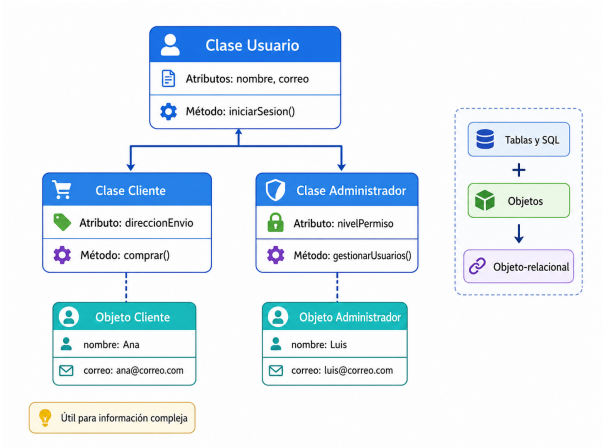
4.4.- Modelo orientado a objetos.

El **modelo orientado a objetos** almacena la información mediante **objetos** que integran datos y comportamiento. Estos objetos se organizan en **clases** y pueden incorporar conceptos propios de la programación orientada a objetos, como **encapsulación, herencia y polimorfismo**.

A diferencia del modelo relacional, que representa la información mediante **tablas**, este modelo utiliza estructuras similares a las empleadas en los lenguajes de **programación orientada a objetos**.

Sus principales elementos son:

1. **Objeto:** representa una entidad concreta y reúne sus **propiedades** y operaciones.
2. **Clase:** define la estructura y comportamiento común de un conjunto de objetos.
3. **Método:** operación asociada a una clase u objeto.



Elaboración propia. *Modelo orientado a objetos.* (CC BY-NC-SA)

Los conceptos fundamentales que incorpora son:

- **Encapsulación:** agrupa los datos y las operaciones que actúan sobre ellos, controlando su acceso.
- **Herencia:** permite crear clases a partir de otras y **reutilizar propiedades y comportamientos**.
- **Polimorfismo:** permite utilizar una misma operación con **comportamientos diferentes** según el objeto.

Ejemplo:

Una aplicación puede definir una clase **Usuario** con atributos como nombre y correo, de la que hereden las clases **Cliente** y **Administrador**, incorporando cada una sus propias características.

Los sistemas **objeto-relacionales** combinan características del modelo relacional con determinados tipos y estructuras más complejas, manteniendo las **tablas y SQL** como base.

El modelo orientado a objetos puede resultar útil cuando se trabaja con información de **estructura compleja**, pero tiene una implantación mucho menor que el **modelo relacional** en las bases de datos de propósito general.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. En el modelo orientado a objetos, una clase define características y comportamientos comunes que pueden compartir varios objetos.

○ Verdadero ○ Falso

Verdadero

La clase actúa como una definición común de **atributos y métodos** para los objetos que pertenecen a ella.

2. La encapsulación permite que una clase herede propiedades y métodos de otra clase.

○ Verdadero ○ Falso

Falso

Esa característica corresponde a la **herencia**. La encapsulación agrupa datos y operaciones y controla el acceso al estado del objeto.

3. El polimorfismo permite que una misma operación pueda comportarse de forma distinta según el objeto sobre el que se aplique.

○ Verdadero ○ Falso

Verdadero

Una misma operación puede tener **implementaciones diferentes** dependiendo del tipo de objeto.

4. Los sistemas objeto-relacionales abandonan completamente las tablas y SQL para trabajar únicamente con objetos.

○ Verdadero ○ Falso

Falso

Combinan características del modelo relacional con estructuras más complejas, manteniendo las **tablas y SQL** como base.

4.5.- Otros modelos.

Además de los modelos **jerárquico, en red, relacional y orientado a objetos**, existen otros modelos diseñados para necesidades concretas. Algunos amplían el modelo relacional y otros, habituales en sistemas **NoSQL**, organizan los datos como **documentos, pares clave-valor, grafos o columnas**, ofreciendo distintas formas de representar y consultar la información.

Existen distintos **modelos de bases de datos** adaptados a diferentes formas de organizar y consultar la información. Los más relevantes son:

Modelo	Características principales	Ejemplo de uso
Objeto-relacional	Amplía el modelo relacional con **tipos de datos complejos**, estructuras definidas por el usuario y otras características avanzadas, manteniendo **tablas y SQL**.	PostgreSQL u Oracle con tipos de datos avanzados.
Documental	Almacena la información en **documentos semiestructurados**, habitualmente en formatos similares a **JSON**.	Catálogos de productos con atributos variables.
Clave-valor	Cada dato se almacena asociado a una **clave única**, facilitando búsquedas muy rápidas cuando se conoce dicha clave.	Sesiones de usuarios o sistemas de caché.
De grafos	Representa la información mediante **nodos y relaciones**, dando especial importancia a las conexiones entre los datos.	Redes sociales, rutas o sistemas de recomendación.
Columnar / familias de columnas	Organiza los datos para gestionar grandes volúmenes y facilitar determinados accesos por **columnas o familias de columnas**.	Análisis de grandes cantidades de datos distribuidos.
Multidimensional	Representa los datos mediante varias **dimensiones y medidas**, facilitando el análisis desde diferentes perspectivas.	Análisis de ventas por **producto, zona y periodo** en sistemas de BI.

Modelo objeto-relacional

El modelo **objeto-relacional** combina la estructura de **tablas** del modelo relacional con características que permiten representar datos más complejos. Los SGBD actuales que incorporan estas funciones continúan utilizando **SQL** como lenguaje principal.

pngegg.com. *MySQL*. (CC BY-NC-SA)

SQL:1999 tuvo importancia histórica en la incorporación de características objeto-relacionales al estándar SQL. Posteriormente, SQL ha continuado evolucionando mediante nuevas revisiones.

Modelos NoSQL

El término **NoSQL** agrupa distintos modelos que no siguen exclusivamente la estructura tradicional de tablas relacionadas. Entre los más utilizados se encuentran los modelos **documentales, clave-valor, de grafos y de familias de columnas**.

No existe un modelo NoSQL mejor para todos los casos: la elección depende de la **estructura de los datos**, las relaciones existentes, el volumen de información y el tipo de consultas necesarias.

Para saber más

Otra forma extendida de clasificar a las bases de datos es en bases de datos sql y NoSql. En el siguiente enlace tienes información sobre bases de datos NoSql: qué son, marcas, tipos y ventajas.

Bases de datos NoSql

Autoevaluación

Una aplicación necesita representar usuarios conectados entre sí mediante amistades, seguidores y recomendaciones, y las consultas se centran principalmente en recorrer esas relaciones.

¿Qué modelo resulta especialmente adecuado?

- a) Clave-valor
- b) Multidimensional

- c) Documental
- d) De grafos

Incorrecta. El modelo **clave-valor** resulta eficiente cuando se conoce la clave del dato, pero no está especialmente diseñado para recorrer relaciones complejas.

Incorrecta. El modelo **multidimensional** está orientado principalmente al **análisis de información** mediante dimensiones y medidas.

Incorrecta. El modelo **documental** resulta adecuado para información semiestructurada y flexible, pero no está especializado en recorrer numerosas relaciones entre elementos.

Correcta. El modelo **de grafos** está diseñado para representar **nodos y relaciones**, por lo que resulta especialmente apropiado cuando las conexiones entre los datos son fundamentales.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

5.- Tipos de bases de datos.

Caso práctico

María pregunta a **Ada** qué tipo de base de datos deberían utilizar para una **nueva aplicación web.**

—Depende de cómo vayamos a utilizar los **datos** —responde **Ada**—. No necesita lo mismo una tienda online que registra operaciones continuamente que un sistema destinado a analizar información histórica.

Juan añade que una misma base de datos puede pertenecer a **varias categorías**, dependiendo del criterio utilizado para clasificarla.

Las **bases de datos** pueden clasificarse siguiendo distintos criterios, como la **variabilidad de la información**, el tipo de procesamiento, su **ubicación** o la forma en que se utilizan. Estas clasificaciones no son excluyentes: una misma base de datos puede ser, por ejemplo, **dinámica, transaccional y distribuida** al mismo tiempo.

Para comprender mejor los distintos tipos de bases de datos, conviene clasificarlas según **cómo cambian sus datos**, **qué tipo de operaciones realizan**, **dónde se almacenan** y **quién puede acceder a ellas**.

Según la variabilidad de los datos

Tipo	Características	Ejemplo
Estáticas	Se utilizan principalmente para **consulta y análisis** de información que apenas cambia.	Archivo histórico de resultados electorales.
Dinámicas	Sus datos pueden **insertarse, modificarse y eliminarse** de forma habitual.	Base de datos de una tienda online con productos, clientes y pedidos.

Según el tipo de procesamiento

Tipo	Características	Ejemplo

Tipo	Características	Ejemplo
Transaccionales (OLTP)	Gestionan numerosas operaciones breves de **inserción, consulta, modificación o eliminación** y necesitan mantener la **consistencia de los datos**.	Compras, reservas o movimientos bancarios.
Analíticas (OLAP)	Están orientadas al **análisis de grandes volúmenes de información**, normalmente histórica, para obtener tendencias y apoyar la toma de decisiones.	Análisis de ventas por año, región y producto.

Según la ubicación de la información

Tipo	Características	Ejemplo
Centralizadas	Los datos se gestionan principalmente desde una **ubicación o sistema central**.	Base de datos de una pequeña empresa alojada en un único servidor.
Distribuidas	Los datos se encuentran repartidos entre **varios nodos o ubicaciones conectadas**, pero forman parte de un mismo sistema.	Empresa con sedes en distintos países que almacenan datos en varios servidores conectados.

Según el ámbito de acceso

Tipo	Características	Ejemplo
Uso interno	El acceso está restringido a determinados **usuarios o aplicaciones** de una organización.	Base de datos interna de empleados de una empresa.
Acceso público	Puede ser consultada por **usuarios externos**, de forma libre o bajo determinadas condiciones.	Base de datos pública de estadísticas oficiales.
Servicio en la nube	La base de datos se ejecuta sobre **infraestructura remota** proporcionada como servicio y accesible mediante red.	Base de datos de una aplicación web desplegada en un proveedor cloud.

Una misma base de datos puede combinar varias características. Por ejemplo, la base de datos de una tienda online puede ser **dinámica**, **transaccional**, de **uso interno** para determinadas operaciones y estar desplegada en la **nube**.

Reflexiona

Una empresa utiliza una base de datos para gestionar **pedidos y pagos en tiempo real**. Además, la dirección analiza los datos acumulados durante varios años para estudiar tendencias de ventas. La base de datos está alojada en varios servidores de distintas sedes y solo puede ser utilizada por personal autorizado.

¿Cómo clasificarías esta base de datos según los distintos criterios estudiados?

Mostrar retroalimentación

La base de datos puede pertenecer a varias categorías al mismo tiempo:

1. **Dinámica**, porque sus datos se **insertan y modifican** continuamente.
2. **Transaccional (OLTP)** para gestionar operaciones como pedidos y pagos.
3. **Analítica (OLAP)** cuando se utilizan los datos históricos para realizar **análisis y detectar tendencias**.
4. **Distribuida**, porque la información se encuentra repartida entre **varios servidores o ubicaciones**.
5. **De uso interno**, porque el acceso está restringido a **usuarios autorizados**.

La clave es entender que estas clasificaciones **no son excluyentes** y describen aspectos diferentes de una misma base de datos.

6.- Sistemas gestores de base de datos.

Caso práctico

Stockbyte (CC BY-NC)

Ada explica a **Juan** y **María** que no basta con diseñar una buena base de datos: también es necesario disponer de un sistema que permita **crear, consultar, modificar y proteger** la información.

—Entonces el SGBD es el intermediario entre las aplicaciones y los datos —comenta **Juan.**

—Exacto —responde **Ada**—. Además de facilitar el acceso, debe controlar la **seguridad, integridad, concurrencia y recuperación** de la información.

Un **Sistema Gestor de Bases de Datos (SGBD o DBMS)** es el software encargado de **definir, almacenar, consultar, modificar y administrar** los datos de una base de datos. Actúa como intermediario entre los **usuarios o aplicaciones** y la información almacenada, aplicando mecanismos de **seguridad, integridad y control de acceso**.

El SGBD evita que cada aplicación tenga que gestionar directamente cómo están almacenados los datos. Proporciona una capa de abstracción que permite trabajar con la información mediante **consultas y operaciones de alto nivel**.

Entre sus principales capacidades se encuentran:

- **Definición de datos:** permite crear tablas, relaciones, restricciones y otras estructuras.
- **Manipulación de datos:** permite realizar operaciones de **consulta, inserción, modificación y eliminación**.
- **Control de acceso:** determina qué **usuarios y aplicaciones** pueden acceder a cada recurso.
- **Integridad:** aplica reglas para mantener los datos **válidos y coherentes**.
- **Concurrencia:** coordina operaciones realizadas simultáneamente por **varios usuarios o procesos**.
- **Recuperación:** proporciona mecanismos de **copias de seguridad, registro y restauración** ante fallos.
- **Independencia de los datos:** reduce la dependencia entre las aplicaciones y la forma física en la que se almacena la información.

El flujo general puede representarse como:

Usuarios / Aplicaciones → SGBD → Sistema de almacenamiento → Datos

El **SGBD** recibe las peticiones, comprueba permisos y restricciones, ejecuta las operaciones necesarias y devuelve los resultados, sin que el usuario tenga que conocer los detalles físicos del almacenamiento.

Elaboración propia. *Sistema Gestor de Bases de Datos* (CC BY-NC-SA)

Principales ventajas

- Reduce la **redundancia e inconsistencia** de la información.
- Mejora la **seguridad y privacidad** mediante usuarios, roles y permisos.
- Facilita el **acceso concurrente** a los datos.
- Permite aplicar reglas de **integridad**.
- Proporciona mecanismos de **recuperación ante fallos**.
- Facilita el acceso mediante lenguajes como **SQL**.
- Separa, en cierta medida, las aplicaciones de la **organización física de los datos**.

Reflexiona

Una empresa dispone de una base de datos utilizada simultáneamente por una tienda online, una aplicación interna de gestión y varios empleados. Durante una operación se produce un fallo en el servidor.

¿Qué responsabilidades debería asumir el SGBD tanto durante el funcionamiento normal como después del fallo?

Mostrar retroalimentación

Durante el funcionamiento normal, el **SGBD** debe controlar los **permisos**, mantener la **integridad de los datos** y coordinar los accesos concurrentes para evitar operaciones incompatibles.

Después de un fallo, debe utilizar sus mecanismos de **recuperación** para restaurar la base de datos a un estado consistente y minimizar la pérdida de información.

6.1.- Funciones.

Un **SGBD** realiza tres funciones fundamentales sobre los datos: **definición**, **manipulación** y **control**. Estas funciones permiten crear la estructura de la base de datos, trabajar con la información almacenada y establecer mecanismos de **seguridad, integridad y acceso**.

Estas funciones se complementan entre sí: la **definición** establece cómo se organiza la base de datos, la **manipulación** permite trabajar con la información y el **control** regula quién puede acceder a ella y bajo qué condiciones.

Función de definición

Permite crear y modificar la **estructura de la base de datos**, definiendo elementos como tablas, atributos, relaciones, restricciones o vistas.

Esta función se realiza mediante el **DDL (Data Definition Language)**.

Algunas operaciones habituales son:

- Crear o eliminar **tablas**.
- Definir **columnas y tipos de datos**.
- Establecer **restricciones**.
- Crear estructuras como **vistas o índices**.

flaticon.es. *Función de definición.* (CC BY-NC-SA)

También permite describir la información en distintos niveles:

1. **Nivel interno:** cómo se almacenan físicamente los datos.
2. **Nivel conceptual:** estructura lógica global de la base de datos.
3. **Nivel externo:** diferentes **vistas** de los datos para usuarios o aplicaciones.

Función de manipulación

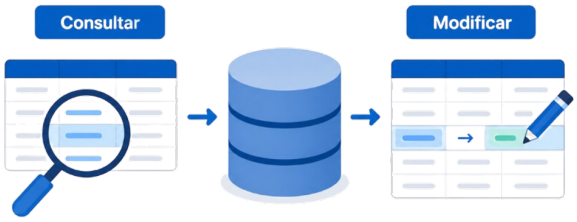
Permite **consultar y modificar** los datos almacenados.

*Se realiza mediante el **DML (Data Manipulation Language)** y comprende principalmente:*

- **Consultar** información.

- **Insertar** nuevos datos.
- **Actualizar** datos existentes.
- **Eliminar** información.

Por ejemplo, una aplicación web puede consultar los productos disponibles, registrar un nuevo pedido o actualizar el stock mediante operaciones DML.



Elaboración propia. *Función de manipulación.* (CC BY-NC-SA)

Función de control

Permite gestionar el **acceso, seguridad e integridad** de la base de datos.

Tradicionalmente estas operaciones se agrupan dentro del **DCL (Data Control Language)** y permiten:

- Crear y gestionar **usuarios y roles**.
- Conceder o retirar **permisos**.
- Limitar el acceso a determinados datos.
- Aplicar mecanismos de **seguridad y auditoría**.

Elaboración propia. *Función de control.* (CC BY-NC-SA)

SQL y estas funciones

El lenguaje **SQL (Structured Query Language)** permite realizar muchas de las operaciones anteriores y suele agruparse de forma didáctica en diferentes subconjuntos:

Lenguaje	Finalidad	Ejemplos de operaciones
DDL	Define la **estructura** de la base de datos.	**CREATE, ALTER, DROP**
DML	Consulta y modifica los **datos**.	**SELECT, INSERT, UPDATE, DELETE**
DCL	Gestiona **permisos y accesos**.	**GRANT, REVOKE**

Autoevaluación

Un administrador está preparando una base de datos para una aplicación de gestión. Durante el proceso realiza las siguientes tareas:

1. Define nuevas **tablas y restricciones**.
2. Consulta y actualiza **registros**.
3. Asigna **permisos** a distintos usuarios.

¿Cuáles de las siguientes afirmaciones son correctas?

- [ ] a) La creación de tablas pertenece a la función de definición.
- [ ] b) La actualización de registros pertenece a la función de control.
- [ ] c) La asignación de permisos pertenece a la función de control.
- [ ] d) La consulta de información forma parte de la función de manipulación.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

6.2.- Componentes.

Un **SGBD** está formado por varios componentes que trabajan de manera coordinada para **definir, almacenar, proteger y utilizar los datos**. Entre los más importantes se encuentran los **lenguajes de la base de datos**, el **diccionario de datos**, el propio **gestor**, los **usuarios** y las **herramientas** de **administración**.

Estos componentes permiten que el sistema pueda ofrecer servicios tanto a las **aplicaciones** como a los distintos perfiles de usuario.

Lenguajes de la base de datos

Permiten definir la estructura de la base de datos, manipular la información y controlar el acceso.

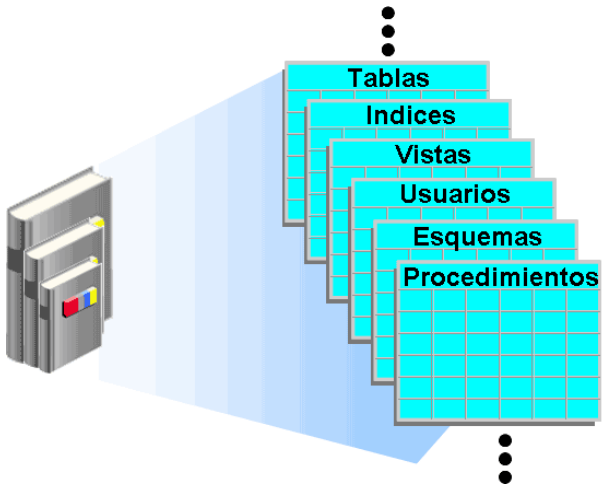
- **DDL:** define estructuras como tablas, relaciones y restricciones.
- **DML:** permite consultar, insertar, actualizar y eliminar datos.
- **DCL:** gestiona permisos y control de acceso.

Diccionario de datos

El **diccionario de datos** almacena los **metadatos** de la base de datos, es decir, información sobre cómo están definidos y organizados los datos.

Puede contener información sobre:

- **Tablas, columnas, vistas e índices**.
- **Tipos de datos y restricciones**.
- **Procedimientos, funciones y disparadores**.
- **Usuarios, roles y permisos**.
- Características de almacenamiento y otros elementos internos del SGBD.



tproduccionmultimedia.wordpress.com. *Diccionario de datos.* (CC BY-NC-SA)

Gestor de la base de datos

Es el componente principal encargado de coordinar el **acceso a los datos** y garantizar que las operaciones se realicen de forma **segura, íntegra y eficiente**.

Entre otras tareas, controla:

- El acceso de **usuarios y aplicaciones**.

Elaboración propia. *Gestor de bases de datos.* (CC BY-NC-SA)

- Las operaciones concurrentes.
- La **seguridad e integridad** de los datos.
- La comunicación con el **sistema operativo y el almacenamiento**.

Usuarios de la base de datos

En un SGBD pueden intervenir distintos perfiles:

Perfil	Funciones principales
DBA	Administra el SGBD, controla **usuarios, permisos, seguridad, copias de seguridad y rendimiento**.
Diseñadores	Definen la **estructura y organización** de la base de datos.
Desarrolladores	Crean aplicaciones que **consultan y modifican** los datos.
Usuarios finales	Acceden a la información mediante aplicaciones o herramientas preparadas para ello.

Herramientas de la base de datos

Los SGBD incluyen o permiten utilizar herramientas para facilitar tareas como:

- Administración de **usuarios y permisos**.
- Diseño y consulta de la base de datos.
- **Copias de seguridad y restauración**.
- Monitorización del **rendimiento**.
- Importación y exportación de datos.

Actividad desplegable

Un equipo está administrando un SGBD y realiza distintas tareas sobre la base de datos. Relaciona cada situación con el componente que interviene principalmente. Lee y completa.

Situación	Componente

Situación	Componente
1. Un administrador consulta qué tablas existen, qué columnas tienen y qué restricciones están definidas.	
2. Una aplicación necesita consultar y modificar información almacenada en la base de datos.	
3. El sistema debe coordinar dos operaciones que intentan modificar los mismos datos al mismo tiempo.	
4. Un técnico realiza una copia de seguridad y supervisa el rendimiento del servidor.	

Comprobar Mostrar retroalimentación

1. **Diccionario de datos:** contiene los **metadatos** que describen tablas, columnas, restricciones, usuarios y otros elementos de la base de datos.
2. **Lenguajes de la base de datos:** permiten **definir, consultar, modificar y controlar** la información mediante instrucciones como las proporcionadas por DDL, DML y DCL.
3. **Gestor de la base de datos:** coordina el **acceso concurrente**, controla las operaciones y ayuda a mantener la **integridad y seguridad** de los datos.
4. **Herramientas de la base de datos:** facilitan tareas de **administración, copias de seguridad, restauración y monitorización**.

6.3.- Arquitectura.

La **arquitectura de un SGBD** organiza la base de datos en distintos niveles para separar **cómo se almacenan los datos**, **cómo se estructura la información** y **qué parte puede ver** cada usuario o aplicación. Esta separación se basa en los niveles **interno, conceptual y externo**, y facilita la **independencia física y lógica** de los datos.

El modelo de referencia más conocido para esta arquitectura es el de **tres niveles ANSI/SPARC**, que distingue cómo se ven los datos desde el almacenamiento, desde el diseño global de la base de datos y desde cada usuario o aplicación.

Niveles de la arquitectura

Nivel	Qué representa	Ejemplo
Interno o físico	Describe **cómo se almacenan físicamente los datos**, incluyendo archivos, estructuras de almacenamiento, índices y métodos de acceso.	Cambiar la organización física de los datos o añadir un índice.
Conceptual o lógico	Define la **estructura global de la base de datos**, incluyendo entidades, atributos, relaciones y restricciones.	Definir las tablas CLIENTES, PEDIDOS y la relación entre ambas.
Externo o de vistas	Define qué parte de la base de datos puede ver cada **usuario o aplicación**.	Un empleado consulta clientes y pedidos, pero no información salarial.

Puede existir un único **esquema interno**, un único **esquema conceptual** y varios **esquemas externos**, adaptados a las necesidades de cada usuario o aplicación.

Independencia de los datos

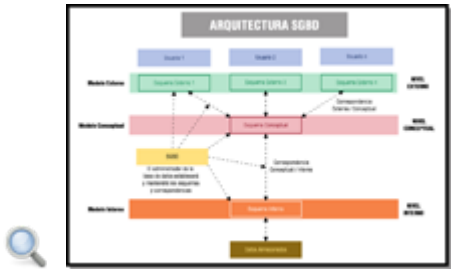
La separación entre niveles permite modificar determinados aspectos de la base de datos sin tener que cambiar todo el sistema:

- **Independencia física:** permite modificar el **nivel interno**, como la organización de archivos o índices, sin alterar el esquema conceptual ni las aplicaciones.
- **Independencia lógica:** permite realizar cambios en el **esquema conceptual** procurando que las vistas externas y las aplicaciones se vean afectadas lo menos posible.

Ejemplo:

Crear un nuevo **índice** para acelerar una consulta modifica la organización física, pero no

obliga a cambiar la aplicación que realiza dicha consulta.



Elaboración propia. *Arquitectura SGBD* (CC BY-NC-SA)

Autoevaluación

Un administrador realiza los siguientes cambios en una base de datos:

1. Añade un **índice** para mejorar el rendimiento de una consulta.
2. Crea una vista para que determinados empleados solo puedan consultar parte de la información.
3. Modifica la estructura lógica incorporando una nueva relación entre dos entidades.

¿Cuál de las siguientes asociaciones es correcta?

- a) Índice → nivel externo; vista → nivel interno; relación → nivel conceptual.
- b) Índice → nivel interno; vista → nivel externo; relación → nivel conceptual.
- c) Índice → nivel conceptual; vista → nivel externo; relación → nivel interno.
- d) Índice → nivel interno; vista → nivel conceptual; relación → nivel externo.

Incorrecta. Un **índice** pertenece al nivel interno y una **vista** representa una perspectiva externa de los datos.

Correcta. El índice afecta al **nivel interno**, la vista pertenece al **nivel externo** y las relaciones forman parte del **nivel conceptual**.

Incorrecta. Una relación entre entidades forma parte del **esquema**

conceptual, no del nivel interno.

Incorrecta. Las vistas se utilizan para definir qué información perciben determinados **usuarios o aplicaciones**, por lo que pertenecen al nivel externo.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

6.4.- Tipos de SGBD.

Los **SGBD** pueden clasificarse según distintos criterios, como el **modelo de datos** que implementan, el **número de usuarios** que pueden trabajar simultáneamente, la **distribución de los datos** o el **propósito** para el que han sido diseñados. Un mismo SGBD puede pertenecer a varias categorías a la vez.

Según el modelo de datos

Tipo	Características	Ejemplo
Relacionales	Organizan los datos en **tablas relacionadas** y utilizan habitualmente SQL.	PostgreSQL, MySQL, Oracle Database.
Jerárquicos	Organizan la información en una estructura de **árbol** con relaciones padre-hijo.	IBM IMS.
En red	Permiten establecer **múltiples relaciones** entre los registros.	IDMS.
Objeto-relacionales	Combinan el modelo relacional con **tipos y estructuras más complejas**.	PostgreSQL u Oracle Database.
NoSQL	Incluyen modelos **documentales, clave-valor, grafos o familias de columnas**.	MongoDB, Redis, Neo4j.

Los modelos **jerárquico y en red** tienen actualmente un uso mucho más reducido, aunque siguen presentes en determinados **sistemas heredados**.

Según el número de usuarios

Tipo	Características	Ejemplo
Monousuario	Está pensado para que trabaje **un único usuario** en un momento determinado.	Base de datos local utilizada por una aplicación personal.
Multiusuario	Permite que **varios usuarios o aplicaciones** accedan simultáneamente, gestionando la concurrencia.	Base de datos de una tienda online.

Según la distribución de los datos

Tipo	Características	Ejemplo
Centralizados	El SGBD y los datos se gestionan principalmente desde un **sistema central**.	Servidor de base de datos de una pequeña empresa.
Distribuidos	Los datos pueden estar repartidos entre **varios nodos conectados** y gestionarse como un único sistema.	Organización con servidores de bases de datos en distintas sedes.

Según su propósito

Tipo	Características	Ejemplo
Propósito general	Pueden utilizarse en aplicaciones de muy distintos ámbitos y tipos de datos.	PostgreSQL, MySQL o SQL Server.
Propósito específico	Están optimizados para una **necesidad o carga de trabajo concreta**.	SGBD especializado en procesamiento analítico o series temporales.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. Un SGBD multiusuario debe gestionar situaciones en las que varias aplicaciones acceden simultáneamente a los datos.

○ Verdadero ○ Falso

Verdadero

Estos sistemas necesitan mecanismos de **concurrencia** que coordinen operaciones simultáneas.

2. Un SGBD distribuido obliga a que todos los datos estén almacenados físicamente en un único servidor.

○ Verdadero ○ Falso

Falso

Los datos pueden encontrarse en **varios nodos o ubicaciones** conectados entre sí.

3. Un SGBD relacional y uno multiusuario son categorías incompatibles entre sí.

○ Verdadero ○ Falso

Falso

Son criterios diferentes: un mismo SGBD puede ser **relacional y multiusuario** simultáneamente.

4. Un SGBD de propósito específico está diseñado u optimizado para determinadas necesidades o cargas de trabajo.

○ Verdadero ○ Falso

Verdadero

Su diseño prioriza un **tipo concreto de utilización**, frente a los sistemas de propósito general.

7.- SGBD comerciales.

Caso práctico

Ada y **Juan** comparan distintos SGBD para una aplicación empresarial que deberá manejar **muchos usuarios**, información crítica y crecer con el tiempo.

—No basta con elegir el más conocido —comenta **Ada**—. Hay que valorar el **rendimiento, la seguridad, la escalabilidad, el soporte técnico y el coste**.

Juan señala que algunos fabricantes ofrecen versiones gratuitas para desarrollo, pero mantienen ediciones y servicios **comerciales** destinados principalmente a entornos profesionales.

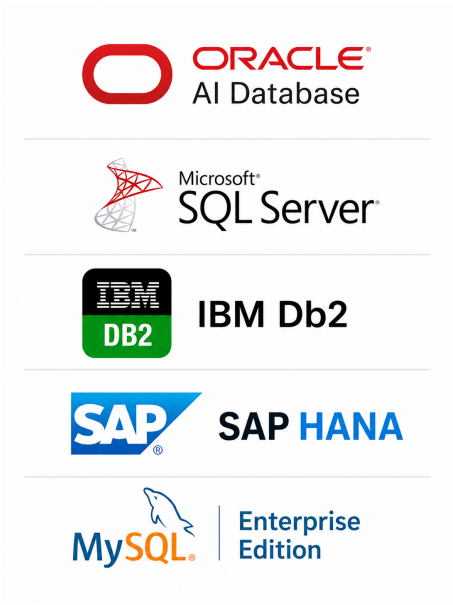
Los **SGBD comerciales** son sistemas cuya utilización profesional puede estar asociada a **licencias, suscripciones o servicios de soporte** ofrecidos por un fabricante. Suelen incorporar herramientas avanzadas de **seguridad, alta disponibilidad, administración y escalabilidad**, especialmente orientadas a entornos empresariales.

La elección de un SGBD no debe basarse únicamente en el precio o en su popularidad. Deben valorarse factores como el **volumen de datos**, número de usuarios, rendimiento, integración con otras tecnologías, disponibilidad, soporte y necesidades de la aplicación.

SGBD	Características principales	Uso habitual	URL
Oracle AI Database	Plataforma empresarial con soporte para **SQL**, alta disponibilidad, seguridad, análisis y diferentes tipos de datos y cargas de trabajo. La versión actual de soporte a largo plazo es **26ai**. (Oracle Docs)	Grandes sistemas empresariales y aplicaciones críticas.	Oracle

SGBD	Características principales	Uso habitual	URL
Microsoft SQL Server	SGBD relacional integrado con el ecosistema Microsoft, con herramientas de **administración, seguridad, análisis y alta disponibilidad**. La versión actual es **SQL Server 2025**. (Microsoft Learn)	Aplicaciones empresariales, servicios web y sistemas basados en tecnologías Microsoft.	MySQL
IBM Db2	SGBD empresarial orientado a cargas de trabajo **transaccionales y analíticas**, disponible en diferentes entornos.	Grandes organizaciones y sistemas empresariales.	IBM Db2
SAP HANA	Plataforma de datos especialmente utilizada en soluciones empresariales SAP, con procesamiento **transaccional y analítico**.	Aplicaciones empresariales y sistemas SAP.	SAP HANA
MySQL Enterprise Edition	Edición comercial de MySQL que añade herramientas y servicios empresariales sobre el conocido SGBD relacional.	Aplicaciones web y sistemas empresariales basados en MySQL.	MySQL Enterprise Edition

Un producto comercial puede disponer también de una **edición gratuita o para desarrollo**. Por ejemplo, Oracle ofrece actualmente **Oracle AI Database Free**, mientras que SQL Server dispone de ediciones gratuitas para determinados usos. Que exista una versión gratuita no convierte necesariamente todo el producto en software libre.



Elaboración propia. *SGBD comerciales.* (CC BY-NC-SA)

Para saber más

Puedes completar más información sobre estos y otros sistemas a través del enlace que te proponemos a continuación:

Sistemas Gestores de Bases de Datos.

Actividad desplegable

Una empresa debe elegir un SGBD comercial para tres proyectos con necesidades diferentes. Selecciona en cada situación la opción más adecuada.

Situación	SGBD
1. Una gran entidad necesita un sistema para una aplicación crítica con alta disponibilidad, seguridad avanzada y soporte empresarial, capaz de manejar un gran volumen de transacciones.	
2. Una empresa desarrolla aplicaciones principalmente con tecnologías Microsoft y busca una solución que se integre con su ecosistema empresarial, herramientas de administración y servicios de datos.	
3. Una organización utiliza soluciones empresariales SAP y necesita combinar procesamiento transaccional y analítico sobre grandes cantidades de información.	

Comprobar Mostrar retroalimentación

1. **Oracle AI Database:** resulta adecuado para entornos empresariales que requieren **alta disponibilidad, seguridad, escalabilidad y procesamiento de grandes cargas de trabajo**.
2. **Microsoft SQL Server:** destaca por su integración con el **ecosistema Microsoft** y por sus herramientas empresariales de

administración, seguridad y análisis.

3. **SAP HANA:** está especialmente orientado a entornos **SAP** y permite combinar cargas **transaccionales y analíticas**.

8.- SGBD libres.

Caso práctico

Juan propone valorar **SGBD libres y de código abierto** para los nuevos proyectos de BK Programación.

—Que un SGBD sea libre no significa que tenga menos capacidad —comenta—. Existen soluciones muy utilizadas en **aplicaciones web, servidores y sistemas empresariales**.

María, que ya ha trabajado con MySQL, quiere comparar distintas alternativas según su **rendimiento, facilidad de despliegue, escalabilidad y tipo de proyecto**.

Los **SGBD libres** son sistemas distribuidos bajo licencias que permiten acceder a su código fuente y, según la licencia, **utilizarlo, modificarlo y redistribuirlo**. Muchos de ellos son ampliamente utilizados en **desarrollo web, aplicaciones empresariales, sistemas embebidos y servicios en la nube**.

No todos los SGBD libres tienen las mismas características. Algunos están orientados a grandes aplicaciones cliente-servidor y otros destacan por su **ligereza, portabilidad o facilidad de integración**.

Principales SGBD libres

SGBD	Características principales	Uso habitual	URL
PostgreSQL	SGBD relacional y objeto-relacional, extensible y con funciones avanzadas de **integridad, concurrencia, JSON y SQL**.	Aplicaciones web, sistemas empresariales y proyectos con consultas complejas.	PostGreSQL
MySQL Community Edition	SGBD relacional muy extendido, especialmente en desarrollo web, con arquitectura **cliente-servidor** y amplio soporte de herramientas y lenguajes.	Aplicaciones web, CMS y servicios online.	MySQL Community Edition

SGBD	Características principales	Uso habitual	URL
MariaDB	SGBD relacional compatible en gran medida con MySQL y desarrollado como proyecto **open source** independiente.	Aplicaciones web y sistemas que utilizan el ecosistema MySQL/ MariaDB.	MariaDB
SQLite	Motor relacional **embebido**, sin servidor independiente y almacenado normalmente en un único archivo.	Aplicaciones móviles, escritorio, pruebas y aplicaciones pequeñas.	SQLite
Firebird	SGBD relacional de código abierto, ligero y con soporte para arquitecturas **embebidas y cliente-servidor**.	Aplicaciones empresariales de pequeño y mediano tamaño.	Firebird

Un SGBD **libre** puede utilizar una licencia que permita estudiar y modificar su código, pero alrededor del producto pueden existir servicios comerciales de **soporte, alojamiento, consultoría o administración**.

Del mismo modo, que una edición de un SGBD sea **gratuita** no implica necesariamente que sea software libre. La diferencia está en las condiciones de su **licencia**, no solo en su precio.

Elaboración propia. *SGBD libres.* (CC BY-NC-SA)

Autoevaluación

Una empresa necesita elegir un SGBD libre para tres proyectos:

1. Una aplicación móvil necesita una base de datos **integrada en la**

propia aplicación, sin mantener un servidor independiente.
2. Una aplicación empresarial requiere consultas complejas, alta integridad y funcionalidades avanzadas de **SQL y JSON**.
3. Un nuevo proyecto web necesita compatibilidad con gran parte del ecosistema y herramientas utilizadas habitualmente con MySQL.

¿Qué asociación es la más adecuada?

- a) 1 PostgreSQL — 2 SQLite — 3 Firebird
- b) 1 SQLite — 2 PostgreSQL — 3 MariaDB
- c) 1 MariaDB — 2 Firebird — 3 SQLite
- d) 1 Firebird — 2 MariaDB — 3 PostgreSQL

Incorrecta. PostgreSQL no está orientado principalmente a funcionar como una base de datos embebida dentro de una aplicación, mientras que SQLite sí.

Correcta. SQLite encaja en aplicaciones embebidas, **PostgreSQL** destaca por sus funcionalidades avanzadas y **MariaDB** mantiene una gran compatibilidad con el ecosistema MySQL.

Incorrecta. SQLite sería más apropiado para el primer escenario y PostgreSQL ofrece más capacidades para el segundo.

Incorrecta. Aunque los tres son SGBD válidos, la asociación no aprovecha las características que distinguen especialmente a cada uno.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

9.- Bases de datos centralizadas.

Caso práctico

Loco085. *Base de datos centralizada.* (CC BY-NC-SA)

Ada, **Juan** y **María** analizan la infraestructura de una empresa que mantiene toda su información en un **sistema central** al que acceden los distintos departamentos.

—Centralizar los datos simplifica bastante la **administración y el control** —comenta **Juan**—, pero ¿qué ocurre si ese sistema deja de estar disponible?

Ada explica que una arquitectura centralizada facilita la gestión, aunque es necesario incorporar mecanismos de **redundancia, copias de seguridad y alta disponibilidad** para reducir el impacto de posibles fallos.

Una **base de datos centralizada** mantiene la gestión de la información en una **ubicación o sistema central**, al que acceden los usuarios y aplicaciones mediante conexiones locales o de red. Esta arquitectura facilita la **administración, seguridad y consistencia** de los datos, aunque concentra una parte importante de la dependencia del sistema en la infraestructura central.

Los usuarios pueden encontrarse en equipos diferentes y acceder mediante una **red**, aunque la base de datos se gestione desde el sistema central.

bbdd.abrilcode.com. *Bases de datos centralizada.* (CC BY-NC-SA)

Sus principales características son:

- La **información** se administra desde una **ubicación central**.
- Facilita la aplicación de **políticas de seguridad, permisos y copias de seguridad**.
- Permite mantener un mayor control sobre la **integridad y consistencia** de los datos.
- Simplifica determinadas tareas de **administración y mantenimiento**.

- Los usuarios y aplicaciones pueden acceder desde **equipos remotos**.
- La disponibilidad depende en gran medida de la **infraestructura central**, por lo que deben existir mecanismos frente a fallos.

Ventajas	Inconvenientes
1. Facilita una **administración centralizada** de los datos.	1. Un fallo grave en la infraestructura central puede afectar a **muchos usuarios y aplicaciones**.
2. Simplifica la gestión de **usuarios, permisos y seguridad**.	2. Puede convertirse en un **cuello de botella** si la infraestructura no tiene capacidad suficiente.
3. Facilita el mantenimiento de la **integridad y consistencia**.	3. Los usuarios remotos dependen de la **conectividad** con el sistema central.
4. Centraliza las **copias de seguridad y recuperación**.	4. El crecimiento puede requerir aumentar los **recursos del sistema central**.
5. Reduce la dispersión y duplicación innecesaria de la información.	5. Requiere medidas de **redundancia y alta disponibilidad** para reducir el impacto de los fallos.

Reflexiona

Una empresa almacena toda su base de datos en un único sistema central. Sus empleados acceden desde distintas oficinas mediante la red. La empresa realiza copias de seguridad diariamente, pero no dispone de ningún sistema alternativo que pueda asumir el servicio si el servidor principal falla.

¿Qué ventaja aporta esta arquitectura y cuál sería su principal riesgo en este caso? ¿Cómo podría reducirse?

Mostrar retroalimentación

La centralización facilita la **administración, seguridad y consistencia** de la información, ya que los datos se gestionan desde un punto común.

El principal riesgo es la **dependencia de la infraestructura central**: si el servidor queda fuera de servicio, los usuarios podrían perder temporalmente el acceso a la base de datos. Las copias de seguridad permiten **recuperar los datos**, pero no garantizan por sí

solas la continuidad inmediata del servicio.

Para reducir este riesgo pueden utilizarse mecanismos de **redundancia, replicación y alta disponibilidad**, capaces de mantener o recuperar rápidamente el servicio ante un fallo.

10.- Bases de datos distribuidas.

Caso práctico

Ada, **Juan** y **María** analizan el sistema de una **cadena hotelera** con sedes en distintas ciudades. Cada hotel necesita trabajar con sus **propios datos**, pero la dirección también debe consultar información conjunta de toda la organización.

—Podríamos repartir la información entre distintos **servidores o nodos** —comenta **Juan**— y permitir que todos formen parte del mismo **sistema de bases de datos**.

Ada explica que una **base de datos distribuida** permite almacenar información en **diferentes ubicaciones** conectadas mediante una **red de comunicaciones**, de forma que los usuarios puedan acceder a ella sin necesitar conocer dónde se encuentra físicamente cada dato.

—Además —añade **María**—, repartir los datos puede ayudar a mejorar la **disponibilidad**, distribuir la **carga de trabajo** y acercar la información a los usuarios que más la utilizan.

Una **base de datos distribuidas (BDD)** está formada por **datos relacionados** almacenados en **varios nodos o ubicaciones** conectados mediante una red, pero gestionados como parte de un **único sistema lógico**. Su objetivo es permitir el acceso a la información con independencia de su **ubicación física**, manteniendo la **coordinación, disponibilidad, transparencia y consistencia** entre los diferentes nodos.

Para diferenciar los principales elementos de un sistema distribuido:

- **Una Base de datos distribuida (BDD):** conjunto de **datos lógicamente relacionados** repartidos entre distintos **nodos conectados mediante una red**.
- **Sistema de bases de datos distribuidas (SBDD):** conjunto formado por las **bases de datos distribuidas**, los nodos y la **red de comunicaciones** que permite trabajar con la información como parte de un mismo sistema.
- **Sistema Gestor de Bases de Datos Distribuidas (SGBDD):** software encargado de **gestionar y coordinar** la BDD, proporcionando acceso a los datos y procurando que su **distribución sea transparente** para usuarios y aplicaciones.

La diferencia puede resumirse así:

BDD = datos distribuidos

SBDD = sistema completo que los integra
SGBDD = software que los gestiona

Cada nodo puede disponer de **almacenamiento**, capacidad de **procesamiento** y una base de datos local, aunque todos colaboran mediante una **red de comunicaciones**.

Ministerio de Educación. *Bases de datos distribuidas* (CC BY-NC-SA)

Una característica fundamental es la **transparencia de distribución**: el usuario o la aplicación puede trabajar con los datos sin tener que conocer exactamente **en qué nodo están almacenados**.

Según las tecnologías utilizadas, los sistemas distribuidos pueden ser:

1. **Homogéneos:** utilizan el mismo **SGBD** o tecnologías compatibles.
2. **Heterogéneos:** integran **SGBD, plataformas o modelos diferentes**, por lo que requieren mayores mecanismos de **integración y coordinación**.

La distribución puede mejorar la **disponibilidad**, el rendimiento y la proximidad de los datos a los usuarios, aunque también aumenta la **complejidad de gestión**.

Ventajas	Inconvenientes
1. Permite distribuir la **carga de trabajo** entre varios nodos.	1. La coordinación entre nodos aumenta la **complejidad** del sistema.
2. Los datos pueden situarse **cerca de los usuarios o aplicaciones** que más los utilizan.	2. Mantener la **consistencia** entre diferentes ubicaciones puede resultar más difícil.
3. Puede mejorar la **disponibilidad** mediante mecanismos de redundancia.	3. Las operaciones dependen de la **red de comunicaciones**.
4. Facilita el crecimiento incorporando nuevos **nodos o recursos**.	4. Las **transacciones distribuidas** son más complejas de gestionar.
5. Los nodos pueden mantener cierta **autonomía local**.	5. La **seguridad, recuperación y monitorización** deben coordinarse entre varios sistemas.

La **replicación** permite mantener **copias de determinados datos** en diferentes nodos para mejorar la disponibilidad o reducir los tiempos de acceso. La forma de **dividir y distribuir los datos** se tratará en el apartado **10.1.- Fragmentación**.

Actividad desplegable

Relaciona cada situación con el concepto que corresponda.

Situación	Concepto
1. Los datos de una empresa están repartidos entre servidores de Madrid, París y Lisboa, pero forman parte de un mismo conjunto lógico.	
2. El conjunto formado por las bases de datos, los nodos y la red que los conecta funciona como un único sistema.	
3. Un software coordina las consultas entre varios nodos y oculta al usuario dónde está almacenado físicamente cada dato.	
4. Una aplicación consulta información sin saber en qué servidor concreto se encuentra almacenada.	

Comprobar Mostrar retroalimentación

1. **BDD:** hace referencia al conjunto de **datos relacionados distribuidos** entre varios nodos.
2. **SBDD:** incluye la **base de datos distribuida**, los nodos y la infraestructura de comunicación que permite tratarlos como un único sistema.
3. **SGBDD:** es el software encargado de **gestionar y coordinar** el acceso a los datos distribuidos.
4. **Transparencia de distribución:** permite que usuarios y aplicaciones trabajen con los datos sin conocer su **ubicación física**.

10.1.- Fragmentación.

La **fragmentación** consiste en dividir una base de datos o una relación en **partes más pequeñas**, llamadas **fragmentos**, que pueden almacenarse en distintos nodos de una base de datos distribuida. Su objetivo es situar los datos **cerca de donde se utilizan**, **reducir transferencias** innecesarias y mejorar el **rendimiento** de determinadas consultas.

El grado de fragmentación debe ser adecuado: una fragmentación demasiado pequeña apenas aporta ventajas, mientras que una excesiva puede aumentar la **complejidad de gestión** y el coste de reconstruir la información.

Para que una fragmentación sea correcta debe cumplir tres propiedades fundamentales:

1. **Completitud:** todos los datos de la relación original deben encontrarse en alguno de los fragmentos.
2. **Reconstrucción:** debe ser posible recuperar la **relación original** combinando correctamente los fragmentos.
3. **Disyunción:** cuando sea aplicable, los fragmentos no deben contener información duplicada innecesariamente. En una fragmentación vertical, la **clave primaria** suele repetirse para permitir posteriormente la reconstrucción.

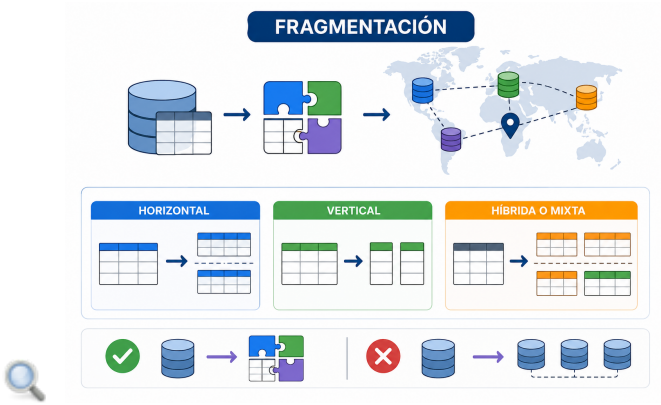
Tipos de fragmentación

Tipo	Cómo divide los datos	Ejemplo
Horizontal	Divide una tabla por **filas o tuplas**, normalmente aplicando una condición.	CLIENTES de España en un nodo y CLIENTES de Francia en otro.
Vertical	Divide una tabla por **columnas o atributos**, conservando normalmente la clave necesaria para relacionar los fragmentos.	Un fragmento con **id_cliente, nombre y email** y otro con **id_cliente, dirección y teléfono**.
Híbrida o mixta	Combina fragmentación **horizontal y vertical** sobre una misma relación.	Separar primero clientes por país y después dividir sus atributos entre distintos fragmentos.

La **fragmentación horizontal** resulta útil cuando distintos nodos trabajan principalmente con subconjuntos de registros, mientras que la **vertical** permite separar grupos de atributos que son utilizados por aplicaciones o usuarios diferentes.

La fragmentación no debe confundirse con la replicación: fragmentar significa dividir los datos, mientras que replicar consiste en mantener copias de los mismos

datos en varios nodos.



Elaboración propia. *Fragmentación.* (CC BY-NC-SA)

Autoevaluación

Una empresa dispone de una tabla CLIENTES con los campos id_cliente, nombre, email, teléfono, país y saldo. Quiere distribuir la información entre varios servidores.

¿Cuáles de las siguientes situaciones representan correctamente una fragmentación?

- [ ] a) Almacenar los clientes de España en un servidor y los de Francia en otro.
- [ ] b) Mantener una copia completa de CLIENTES en tres servidores diferentes.
- [ ] c) Guardar id_cliente, nombre y email en un fragmento, y id_cliente, teléfono y saldo en otro.
- [ ] d) Dividir primero los clientes por país y después separar algunos de sus atributos.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

11.- Primeros pasos en Oracle Database.

Oracle Database es un **Sistema Gestor de Bases de Datos (SGBD) relacional y objeto-relacional** desarrollado por Oracle. Está orientado a **almacenar**, **consultar** y **administrar** grandes volúmenes de **información,** incorporando funciones de **seguridad, integridad, concurrencia, recuperación y alta disponibilidad** para aplicaciones de distinto tamaño.

Oracle AI Database 26ai Free es la edición gratuita actual de Oracle orientada al **aprendizaje, desarrollo y pruebas**. Permite trabajar con una base de datos Oracle real, utilizar **SQL**, crear usuarios y familiarizarse con herramientas como **SQL*Plus** o **SQLcl** sin necesidad de adquirir una licencia comercial.

cleanpng.com. *Logotipo Oracle Database.* (CC BY-NC-SA)

Oracle ofrece versiones de **Oracle AI Database 26ai Free** para **Windows y Linux**. En Windows, la instalación se realiza descargando el paquete oficial, descomprimiéndolo y ejecutando el instalador.

Durante la configuración se crea una arquitectura básica formada por:

Elemento	Función
FREE	Base de datos contenedora o **CDB** creada por defecto.
FREEPDB1	Base de datos conectable o **PDB** en la que pueden crearse usuarios y objetos para las prácticas.
Listener	Servicio que permite establecer **conexiones** con la base de datos. Utiliza por defecto el puerto **1521**.
SYS / SYSTEM	Usuarios administrativos utilizados para tareas de **gestión y configuración**.

La instalación de Oracle AI Database Free crea por defecto la base contenedora **FREE** y la PDB **FREEPDB1**.

Conexión a la base de datos

Oracle permite utilizar diferentes herramientas. Para comenzar pueden emplearse **SQL*Plus** o **SQLcl**, ambas capaces de ejecutar instrucciones SQL desde una terminal.

Una conexión a la PDB puede realizarse indicando:

```
usuario@servidor:puerto/servicio
```

Ejemplo:

```
sqlplus sys@localhost:1521/FREEPDB1 as sysdb
```

Aquí:

- **localhost** indica que Oracle se ejecuta en el mismo equipo.
- **1521** es el puerto utilizado por defecto.
- **FREEPDB1** es el servicio de la PDB.
- **SYSDBA** concede privilegios administrativos durante la conexión.

Para las prácticas habituales no conviene trabajar continuamente con cuentas administrativas. Lo adecuado es crear un **usuario propio** y concederle únicamente los privilegios necesarios.

Actividad desplegable

Relaciona cada elemento de Oracle con su función.

Elemento	Función
	Base de datos contenedora o **CDB** creada por defecto.
	Base de datos conectable o **PDB** utilizada para trabajar con usuarios y objetos.
	Servicio encargado de recibir las **conexiones** hacia la base de datos.
	Privilegio utilizado para realizar tareas de **administración**.

Comprobar Mostrar retroalimentación

1. **FREE:** corresponde a la **CDB** creada por defecto en Oracle AI Database Free.
2. **FREEPDB1:** es la **PDB** en la que pueden crearse usuarios y objetos para las prácticas.
3. **Listener:** gestiona las **conexiones de clientes** hacia Oracle.
4. **SYSDBA:** proporciona privilegios elevados para realizar tareas de

administración.

12.- Legislación.

Caso práctico

Una empresa quiere crear una base de datos con información de sus **clientes** para gestionar pedidos, facturación y comunicaciones comerciales.

Antes de almacenar los datos, debe determinar **qué información necesita**, para qué finalidad se utilizará, durante cuánto tiempo se conservará y **quién podrá acceder** a ella.

freepik. *Legislación* (CC BY-NC-SA)

Además, deberá aplicar desde el diseño medidas que garanticen la **privacidad y seguridad**, informar correctamente a las personas afectadas y disponer de una **base jurídica válida** para tratar sus datos.

El tratamiento de **datos personales** mediante bases de datos debe respetar la normativa de protección de datos y aplicar medidas **técnicas y organizativas** adecuadas. En España, las referencias principales son el **RGPD** y la **LOPDGDD**, que regulan principios, derechos, responsabilidades y obligaciones relacionadas con la recogida, almacenamiento y utilización de información personal.

Normativa principal

Norma	Estado y ámbito principal
RGPD – Reglamento (UE) 2016/679	Norma europea principal sobre **protección y tratamiento de datos personales**.
LOPDGDD – Ley Orgánica 3/2018, de 5 de diciembre	Complementa y adapta el RGPD al ordenamiento español y regula también los **derechos digitales**. (BOE)

Norma	Estado y ámbito principal
Ley Orgánica 7/2021, de 26 de mayo	Regula específicamente los datos tratados para **prevención, detección, investigación y enjuiciamiento de infracciones penales** y ejecución de sanciones penales. (BOE)
ENS – Real Decreto 311/2022, de 3 de mayo	Regula el **Esquema Nacional de Seguridad**. (BOE)
LSSI – Ley 34/2002, de 11 de julio	Regula los **servicios de la sociedad de la información y el comercio electrónico**. (BOE)

La **LOPDGDD** continúa vigente y su texto consolidado incorpora modificaciones posteriores. El **ENS** vigente está regulado por el Real Decreto 311/2022.

Principios de protección de datos

*Cuando una base de datos contiene **datos personales**, el tratamiento debe respetar principios como:*

vectorjuice. *Protección de datos.* (CC BY-NC-SA)

- **Licitud, lealtad y transparencia:** los datos deben tratarse legalmente y de forma clara para la persona interesada.
- **Limitación de la finalidad:** solo deben utilizarse para los **fines determinados** para los que fueron recogidos.
- **Minimización de datos:** deben solicitarse únicamente los datos **necesarios**.
- **Exactitud:** la información debe mantenerse **correcta y actualizada**.
- **Limitación del plazo de conservación:** los datos no deben conservarse más tiempo del necesario.
- **Integridad y confidencialidad:** deben protegerse frente a **accesos, pérdidas o modificaciones no autorizadas**.
- **Responsabilidad proactiva:** la organización debe poder demostrar que aplica correctamente estas obligaciones.
- Roles principales

Rol	Función
Interesado	Persona física a la que pertenecen los **datos personales**.
Responsable del tratamiento	Decide **para qué y cómo** se tratan los datos.
Encargado del tratamiento	Trata datos personales **por cuenta del responsable**.

Rol	Función
Delegado de Protección de Datos (DPD/DPO)	Supervisa y asesora sobre el cumplimiento de la normativa cuando su designación resulta obligatoria o se realiza voluntariamente.

Derechos de las personas

Las personas pueden ejercer, cuando correspondan, derechos como:

- **Acceso**
- **Rectificación**
- **Supresión**
- **Limitación del tratamiento**
- **Portabilidad**
- **Oposición**
- Derecho a no ser objeto de determinadas **decisiones exclusivamente automatizadas**.

Seguridad y brechas de datos

Una **brecha de datos personales** puede producirse por pérdida, destrucción, alteración o **acceso no autorizado** a información personal.

Cuando sea probable que la brecha suponga un **riesgo para los derechos y libertades** de las personas, el responsable debe notificarla a la autoridad de control, cuando sea posible, dentro de las **72 horas** desde que tenga constancia. Si existe un **alto riesgo**, además puede ser necesario comunicarla a las personas afectadas.

Reflexiona

Una tienda online recoge de sus clientes **nombre, dirección, teléfono, correo electrónico, DNI, fecha de nacimiento y profesión**. Sin embargo, para gestionar un pedido solo necesita parte de esa información. Además, todos los empleados pueden consultar la base de datos completa y los datos se conservan indefinidamente.

¿Qué principios de protección de datos crees que debería revisar la empresa y qué medidas debería aplicar para cumplir mejor con la normativa?

Mostrar retroalimentación

La empresa debería revisar especialmente:

1. **Minimización de datos:** solicitar únicamente la información necesaria para la finalidad prevista.
2. **Limitación de la finalidad:** utilizar los datos solo para los fines para los que fueron recogidos.
3. **Limitación del plazo de conservación:** no conservar la información durante más tiempo del necesario.
4. **Integridad y confidencialidad:** limitar el acceso a los datos según las funciones de cada usuario y aplicar medidas de **seguridad** adecuadas.

Por tanto, debería reducir los datos solicitados, definir durante cuánto tiempo se conservarán y establecer **usuarios, roles y permisos** para que cada empleado acceda únicamente a la información que necesite.

13.- Big Data e Inteligencia de negocio

Caso práctico

DARPA. *Código binario.* (CC BY-NC-SA)

Una empresa recopila un **gran volumen de datos** procedentes de clientes, ventas, aplicaciones web, sensores y redes sociales. El objetivo no es solo almacenarlos, sino analizarlos para detectar **patrones, tendencias y oportunidades** que ayuden a mejorar la toma de decisiones.

Para ello necesita tecnologías capaces de gestionar datos **estructurados, semiestructurados y no estructurados**, además de herramientas que permitan transformarlos en **información útil para el negocio**.

Big Data engloba tecnologías y técnicas destinadas a **almacenar**, **procesar** y **analizar** grandes volúmenes de **datos**, especialmente cuando presentan gran **variedad, velocidad o complejidad**. La **Inteligencia de Negocio (Business Intelligence, BI)** utiliza esos datos para generar **información útil, indicadores y análisis** que faciliten la toma de decisiones.

La información puede proceder de muchas fuentes:

- **Bases de datos** internas o externas.
- Ficheros **CSV, JSON o XML**.
- Aplicaciones web y registros de actividad.
- **Sensores e IoT**.
- Servicios y plataformas externas.

ctisoluciones.com. *Big data.* (CC BY-NC-SA)

Antes de analizar los datos, normalmente es necesario **integrarlos, limpiarlos y transformarlos** para garantizar su calidad.

Tipos de análisis

Tipo	Objetivo	Ejemplo

Tipo	Objetivo	Ejemplo
Descriptivo	Explica **qué ha ocurrido**.	Ventas obtenidas durante el último trimestre.
Predictivo	Estima **qué podría ocurrir** utilizando datos históricos y modelos.	Predecir la demanda de un producto.
Prescriptivo	Propone **qué acciones realizar** para alcanzar un objetivo.	Recomendar cuánto stock pedir según la demanda prevista.

Estos análisis permiten identificar **tendencias**, detectar comportamientos relevantes y apoyar decisiones basadas en datos.

Tecnologías de almacenamiento y procesamiento

Tecnología	Características	Ejemplo de uso
Almacenamiento distribuido	Reparte grandes cantidades de datos entre **varios nodos**, facilitando la escalabilidad y la tolerancia a fallos.	HDFS o almacenamiento de objetos en la nube.
Bases de datos NoSQL	Permiten trabajar con datos **semiestructurados o no estructurados** y escalar horizontalmente en determinados escenarios.	MongoDB, Cassandra o bases de datos de grafos.
Procesamiento distribuido	Permite procesar grandes conjuntos de datos utilizando **varios nodos en paralelo**.	Apache Spark.
Data Warehouse	Almacena datos integrados e históricos optimizados para **análisis y BI**.	Análisis de ventas, clientes y productos.
Data Lake	Conserva grandes cantidades de datos en su formato original o poco transformado.	Datos de logs, sensores, documentos y eventos.

Apache Spark es principalmente un motor de procesamiento distribuido que puede aprovechar la memoria para acelerar determinadas operaciones. En arquitecturas actuales también son habituales los conceptos de **Data Warehouse y Data Lake**.

Big Data e Inteligencia de negocio

Aunque están relacionados, no son exactamente lo mismo:

1. **Big Data** se centra en gestionar y procesar grandes cantidades y variedades de datos.
2. **Business Intelligence (BI)** se centra en **analizar y presentar información** mediante informes, indicadores y cuadros de mando para apoyar decisiones.

Ejemplo:

Una empresa puede almacenar millones de registros de ventas y comportamiento de clientes mediante tecnologías Big Data y utilizar posteriormente una herramienta de **BI** para mostrar las ventas por zona, detectar tendencias y comparar resultados.

Autoevaluación

Una empresa recopila millones de eventos de su aplicación web, datos de ventas y registros de sensores. Después quiere analizar esa información para prever la demanda y mostrar indicadores de negocio a la dirección.

¿Cuáles de las siguientes afirmaciones son correctas?

- [ ] a) El análisis que intenta prever la demanda es descriptivo.
- [ ] b) Un Data Lake puede utilizarse para almacenar grandes cantidades de datos de distintos formatos.
- [ ] c) Apache Spark es principalmente una tecnología de almacenamiento de datos.
- [ ] d) Apache Spark puede utilizarse para procesar grandes conjuntos de datos de forma distribuida.

Mostrar retroalimentación

Solución

1. Incorrecto
2. Correcto
3. Correcto
4. Correcto

Licencia: licencia propietaria